

Exploratory Analysis of Rain Fall Data in India for Agriculture

CHAPTER - 01

Project Abstract:

The project, "Exploratory Analysis of Rainfall Data in India for Agriculture," is a comprehensive data-driven study aimed at decoding the complexities of the Indian Monsoon and its direct consequences on the nation's primary sector. Agriculture in India is not merely an industry; it is a socio-economic backbone that supports over 50% of the population. However, it remains heavily dependent on the "vagaries of the monsoon." This project utilizes historical rainfall data spanning over a century to identify patterns, shifts, and anomalies that influence crop cycles, irrigation needs, and food security.

Agriculture is the primary source of livelihood for about 58% of India's population. The success of agricultural output is heavily dependent on the monsoons, making rainfall a crucial climatic factor. However, climate change has led to erratic rainfall patterns, causing floods, droughts, and crop failures. This project, titled "Exploratory Analysis of Rain Fall Data in India for Agriculture", focuses on analyzing historical rainfall data to provide actionable insights.

Using Python and Data Analytics libraries such as Pandas, Matplotlib, and Seaborn, this project performs Exploratory Data Analysis (EDA) on a dataset covering monthly and annual rainfall across various states and sub-divisions of India over the last century. The system visualizes trends, seasonal variations, and rainfall distribution patterns. The outcome of this analysis helps in understanding the climatic shifts and provides valuable information for farmers, policymakers, and agricultural scientists to plan crop cycles and water management strategies effectively.

Exploratory Analysis of Rain Fall Data in India for Agriculture

1.1 - Project Overview

India is an agricultural country where the economy is deeply intertwined with the monsoon season. Rainfall is the most significant parameter influencing crop production, water resources, and power generation. This project aims to perform a detailed statistical and visual analysis of India's rainfall data. By leveraging data science techniques, we intend to uncover hidden patterns and trends that are not visible through manual observation of raw data tables.

Geographical and Temporal Scope: The scope of this analysis is comprehensive, covering both spatial and temporal dimensions: Plateau.

Spatial Scope: The study covers all major States and Union Territories of India, with a specific focus on high-yield agricultural belts like the Indo-Gangetic Plains and the Deccan

Temporal Scope: The dataset spans from the early 20th century (1901) to the modern era (2024). This long-term view helps in distinguishing between standard weather fluctuations and genuine climate change trends.

Seasonal Segmentation: The analysis breaks down rainfall into four distinct meteorological seasons: Winter (Jan-Feb), Pre-Monsoon (Mar-May), South-West Monsoon (Jun-Sep), and Post-Monsoon (Oct-Dec).

Flood Warning Systems: Understanding the trend of "Extreme Weather Events" helps local administrations in disaster preparedness and urban planning to prevent waterlogging and crop destruction.

Exploratory Analysis of Rain Fall Data in India for Agriculture

1.2 - Purpose of the Project:

Primary Purpose: The primary purpose of this project is to transition from intuition-based farming to Data- Driven Agriculture. While meteorological departments provide raw data, it often remains inaccessible or unintelligible to the agricultural sector. This project aims to bridge that gap by performing a rigorous Exploratory Data Analysis (EDA) on historical rainfall datasets. The goal is to uncover hidden patterns, quantify climate variability, and provide actionable insights that can assist in crop planning, water resource management, and risk assessment for the Indian agrarian economy.

To Perform Spatio - Temporal Analysis: To Evaluate Seasonal Variability: To analyze how rainfall patterns have shifted over time (Temporal) across different Indian states and districts (Spatial). This involves comparing the data of the pre-green revolution era (early 1900s) with modern climate trends (2000–2025).

To Evaluate Seasonal Variability: To deconstruct annual rainfall into seasonal components—Winter, Pre-Monsoon, Monsoon, and Post-Monsoon. The objective is to assess the reliability of the South-West Monsoon, which determines the fate of Kharif crops, and the North-East Monsoon, which is critical for Rabi crops in southern India.

Optimization of Water Resources Irrigation Scheduling: By identifying the frequency and duration of "Dry Spells" (breaks in the monsoon), irrigation departments can plan water releases from dams more effectively to save crops during critical stress periods.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE
MAXIMUM MARKS	2 MARKS

CHAPTER – 02

2.1 – PROBLEM STATEMENT:

Indian agriculture highly depends on rainfall. Unpredictable rainfall patterns cause droughts, floods, and crop damage. There is a need to analyze rainfall data to understand patterns and support better agricultural planning.

Farmers face uncertainty due to:

- Irregular rainfall patterns
- Delayed monsoon
- Drought and flood conditions
- Lack of accurate rainfall insights

Traditional decision-making based on assumptions leads to crop loss and financial instability. Therefore, a data-based analytical system is required to reduce uncertainty.

1. Background of the Problem

Agriculture is one of the most important sectors in India and supports a large portion of the population. A significant percentage of Indian farming depends directly on rainfall, especially the monsoon season. Any variation in rainfall patterns directly affects crop growth, yield, and farmers' income.

In recent years, rainfall has become increasingly unpredictable due to climate change and environmental variations. This unpredictability has created serious challenges for agricultural planning and productivity.

2. Core Problem

The major problem is the uncertainty in rainfall distribution across different regions and seasons in India. Farmers are unable to accurately predict:

- The amount of rainfall
- The timing of monsoon arrival
- The duration of rainfall

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- The occurrence of droughts or floods

This uncertainty leads to improper crop planning and resource management.

3. Challenges Faced by Farmers

Farmers face several difficulties due to irregular rainfall patterns:

- Irregular and uneven rainfall distribution
- Delayed or early monsoon
- Sudden drought and flood conditions
- Crop damage due to excessive rainfall
- Water scarcity during critical crop growth stages
- Financial instability caused by crop loss

Most agricultural decisions are made based on traditional knowledge and assumptions rather than structured data analysis.

4. Impact of the Problem

Unpredictable rainfall results in:

- Reduced agricultural productivity
- Crop failure and yield loss
- Increased debt burden among farmers
- Economic instability in rural areas
- Food supply fluctuations

Without proper rainfall analysis, it becomes difficult to identify long-term trends and regional variations.

5. Need for a Solution

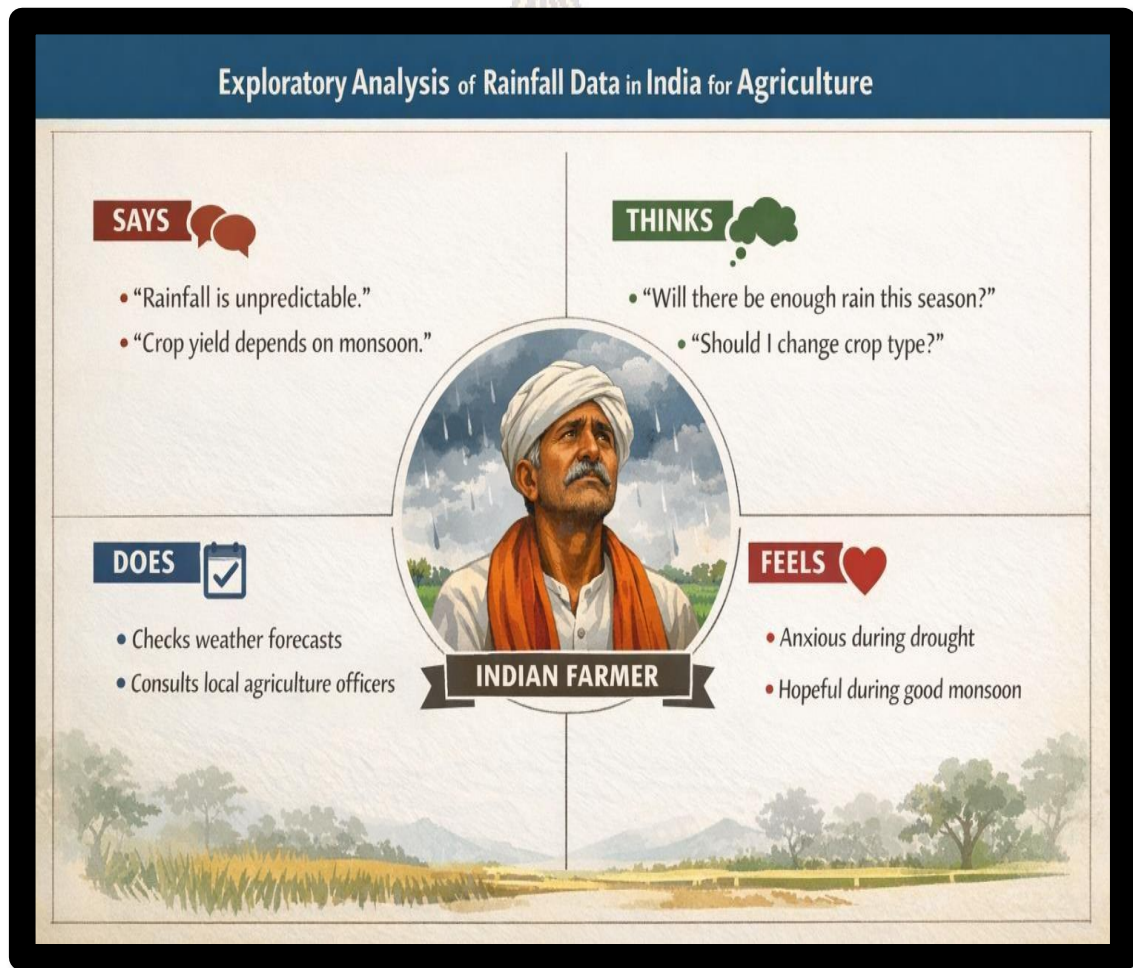
There is a strong need for a data-driven approach to analyze historical rainfall data and identify meaningful patterns. By performing exploratory analysis, it becomes possible to:

- Understand seasonal rainfall trends
- Detect drought-prone and flood-prone areas
- Support informed agricultural planning
- Reduce uncertainty in farming decisions

Exploratory Analysis of Rain Fall Data in India for Agriculture

DATE	28-02-2026
TEAM ID	LTVIP2026TMID89121
PROJECT NAME	EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE
MAXIMUM MARKS	4 MARKS

2.2 - Empathy Map Canvas:



Gayatri Educational Society
ESTD-2001

Exploratory Analysis of Rain Fall Data in India for AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	4 MARKS

2.3 – Brainstorming:

Brainstorming provides a free and open environment that encourages everyone within a team to participate in the creative thinking process that leads to problem solving. Prioritizing volume over value, out-of-the-box ideas are welcome and built upon, and all participants are encouraged to collaborate, helping each other develop a rich amount of creative solutions.

Use this template in your own brainstorming sessions so your team can unleash their imagination and start shaping concepts even if you're not sitting in the same room.

Reference: <https://www.mural.co/templates/brainstorm-and-idea-prioritization>

Step-1: Team Gathering, Collaboration and Select the Problem Statement



Brainstorm & idea prioritization

Use this template in your own brainstorming sessions so your team can unleash their imagination and start shaping concepts even if you're not sitting in the same room.

🕒 10 minutes to prepare
🕒 1 hour to collaborate
👤 2-8 people recommended

➡️

Before you collaborate
A little bit of preparation goes a long way with this session. Here's what you need to do to get going.

🕒 10 minutes

A Team gathering
Define who should participate in the session and send an invite. Share relevant information or pre-work ahead.

B Set the goal
Think about the problem you'll be focusing on solving in the brainstorming session.

C Learn how to use the facilitation tools
Use the Facilitation Superpowers to run a happy and productive session.

[Open article](#) ➡️

1

Define your problem statement
What problem are you trying to solve? Frame your problem as a How Might We statement. This will be the focus of your brainstorm.

🕒 5 minutes

PROBLEM
How might we [your problem statement]?

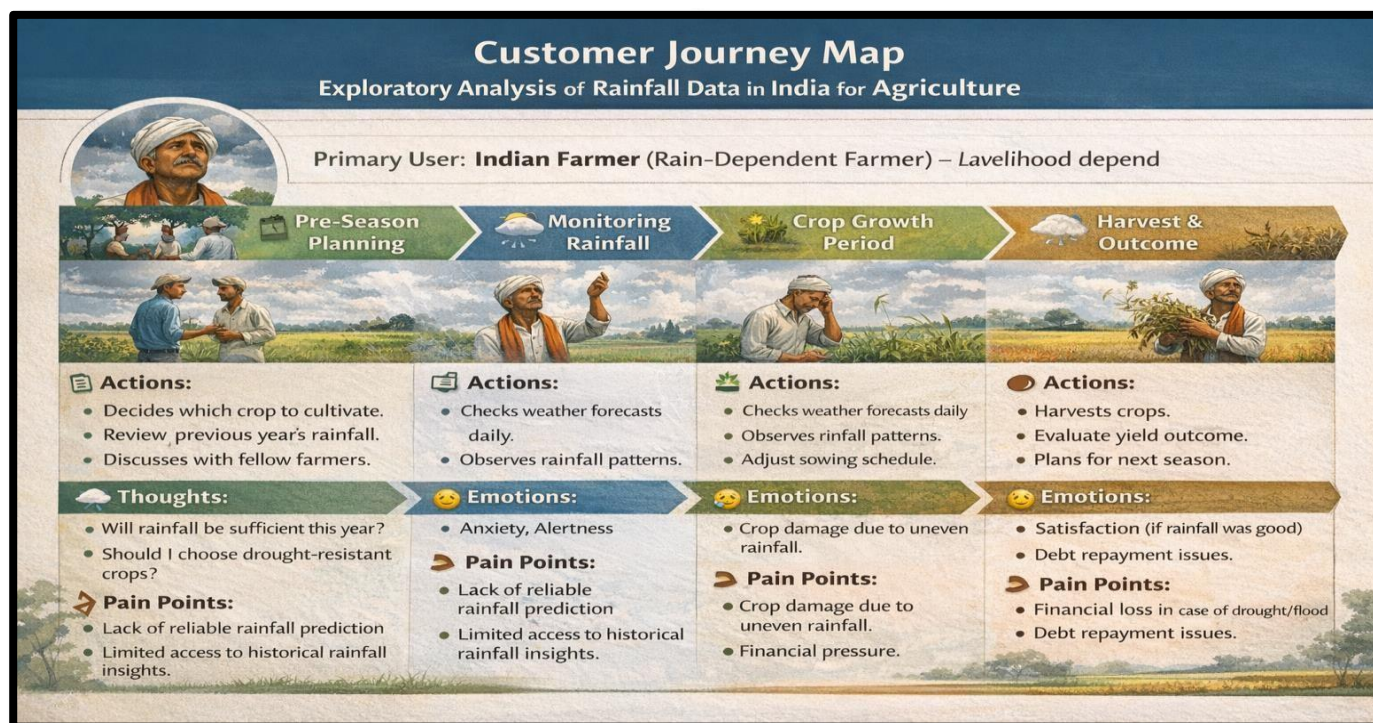
Key rules of brainstorming
To run a smooth and productive session

- Stay in topic.
- Defer judgment.
- Go for volume.
- Encourage wild ideas.
- Listen to others.
- If possible, be visual.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE
MAXIMUM MARKS	2MARKS

3.1 - Customer Journey map:



JOURNEY STAGES

The agricultural journey is divided into four major stages:

Stage 1: Pre-Season Planning

Actions:

- Decides which crop to cultivate.
- Reviews previous year's rainfall data.
- Discusses with fellow farmers.

Thoughts:

- Will rainfall be sufficient this year?
- Should I choose drought-resistant crops?

Pain Points:

- Lack of reliable rainfall prediction.
- Limited access to historical rainfall insights.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

At this stage, uncertainty begins due to unpredictable monsoon patterns.

Stage 2: Monitoring Rainfall

Actions:

- Checks weather forecasts daily.
- Observes rainfall patterns.

Emotions:

- Anxiety
- Alertness

Pain Points:

- Inaccurate or delayed rainfall predictions.
- Limited data-driven guidance.

The farmer closely monitors rainfall conditions to adjust farming activities accordingly.

Stage 3: Crop Growth Period

Actions:

- Continues checking forecasts.
- Observes crop condition.
- Adjusts sowing or irrigation schedule.

Emotions:

- Concern
- Stress

Pain Points:

- Crop damage due to uneven rainfall.
- Financial pressure due to uncertain yield.

Rainfall irregularities during this stage directly affect crop health and productivity.

Stage 4: Harvest & Outcome

Actions:

- Harvests crops.
- Evaluates yield outcome.
- Plans for next season.

Emotions:

- Satisfaction (if rainfall was good)
- Disappointment (if rainfall was poor)

Pain Points:

- Financial loss in case of drought or flood.
- Debt repayment issues.

The final outcome depends heavily on rainfall distribution throughout the season.

4. Importance of the Customer Journey Map

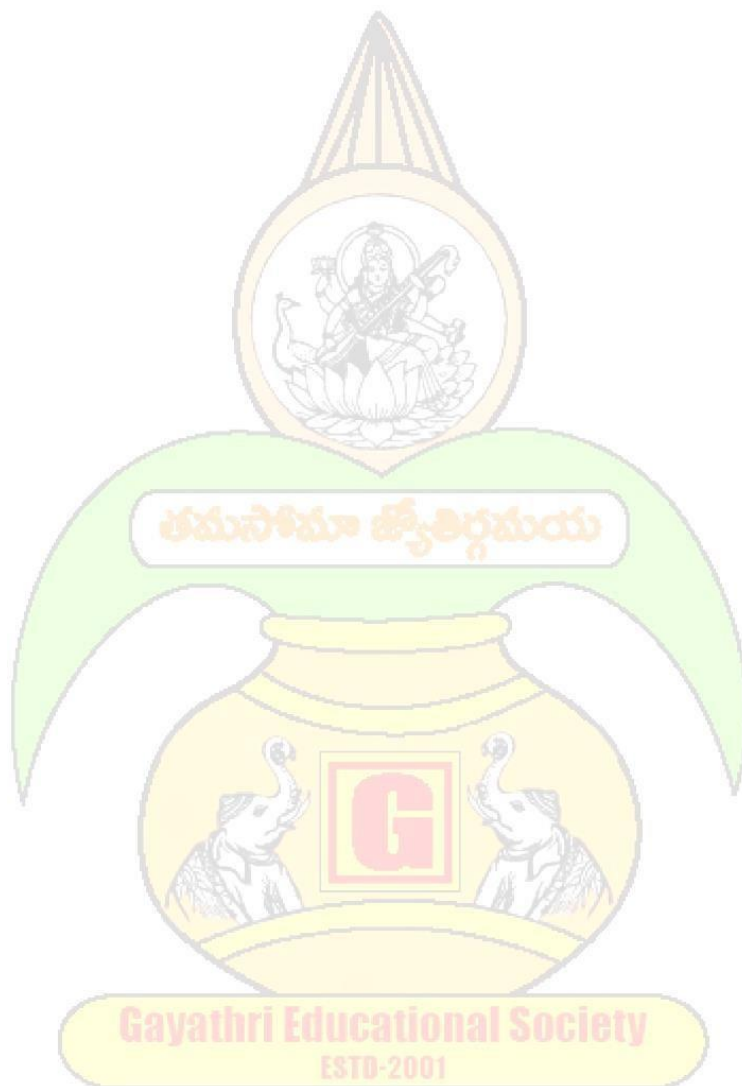
The Customer Journey Map helps to:

- Understand farmer behavior across the agricultural cycle.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Identify critical pain points related to rainfall.
- Connect rainfall data analysis with real-world challenges.
- Support development of a data-driven analytical solution.

By analyzing rainfall trends and seasonal patterns, the project aims to reduce uncertainty at every stage of the farmer's journey.



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	2 MARKS

3.2 - Solution Requirement

The Solution Requirement section describes the features, functionalities, and technical needs of the proposed rainfall data analysis system. The purpose of this system is to analyze historical rainfall data and generate meaningful insights that can support agricultural planning and reduce uncertainty for farmers.

Functional Requirements

Functional requirements describe what the system should perform.

Data Collection

- The system should collect historical rainfall data (monthly/yearly).
- Data should include region-wise or state-wise rainfall records

Data Preprocessing

- The system should clean missing or inconsistent data.
- It should handle null values and incorrect entries.
- Data should be structured for analysis.

Exploratory Data Analysis (EDA)

- Generate summary statistics (mean, median, variance).
- Analyze seasonal rainfall distribution.
- Compare rainfall across different regions.
- Identify trends and patterns over time.

Data Visualization

- Create line charts for yearly rainfall trends.
- Create bar charts for region-wise rainfall comparison.
- Generate heatmaps or seasonal distribution graphs.
- Present results in a clear and understandable format.

Insight Generation

- Identify drought-prone areas.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Identify flood-prone regions.
- Provide observations that support agricultural planning.

Non-Functional Requirements

Non-functional requirements define system quality and performance.

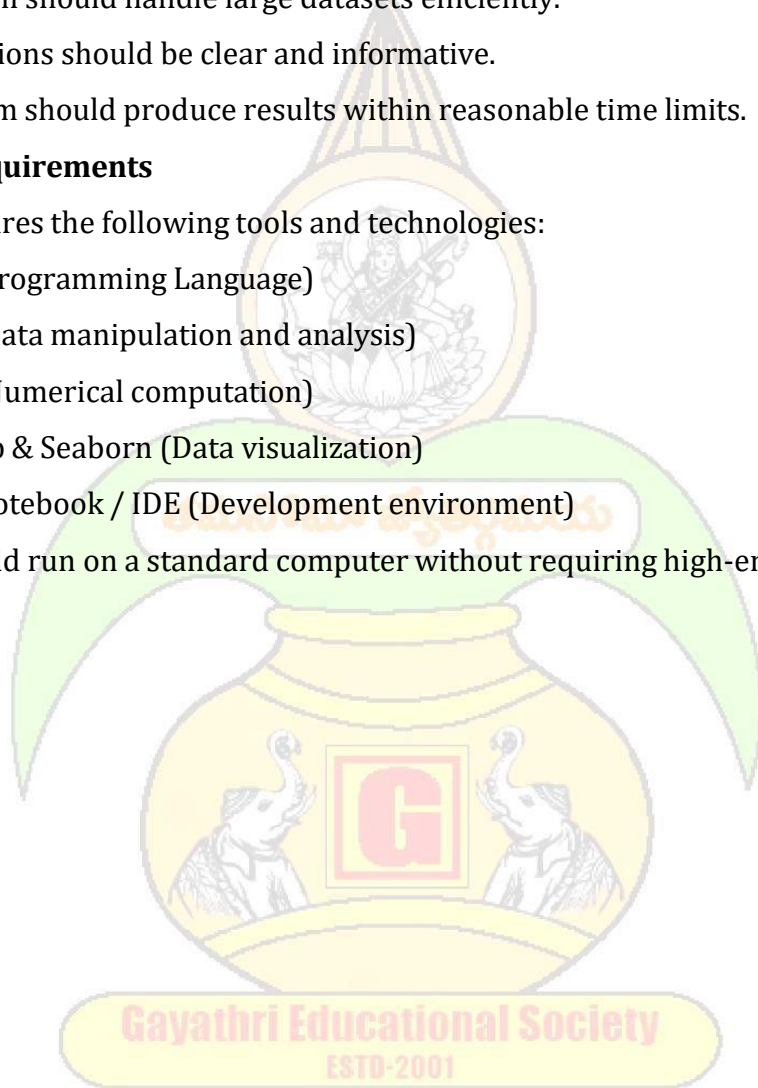
- The system should be user-friendly and easy to interpret.
- The analysis should be accurate and reliable.
- The system should handle large datasets efficiently.
- Visualizations should be clear and informative.
- The system should produce results within reasonable time limits.

5. Technical Requirements

The project requires the following tools and technologies:

- Python (Programming Language)
- Pandas (Data manipulation and analysis)
- NumPy (Numerical computation)
- Matplotlib & Seaborn (Data visualization)
- Jupyter Notebook / IDE (Development environment)

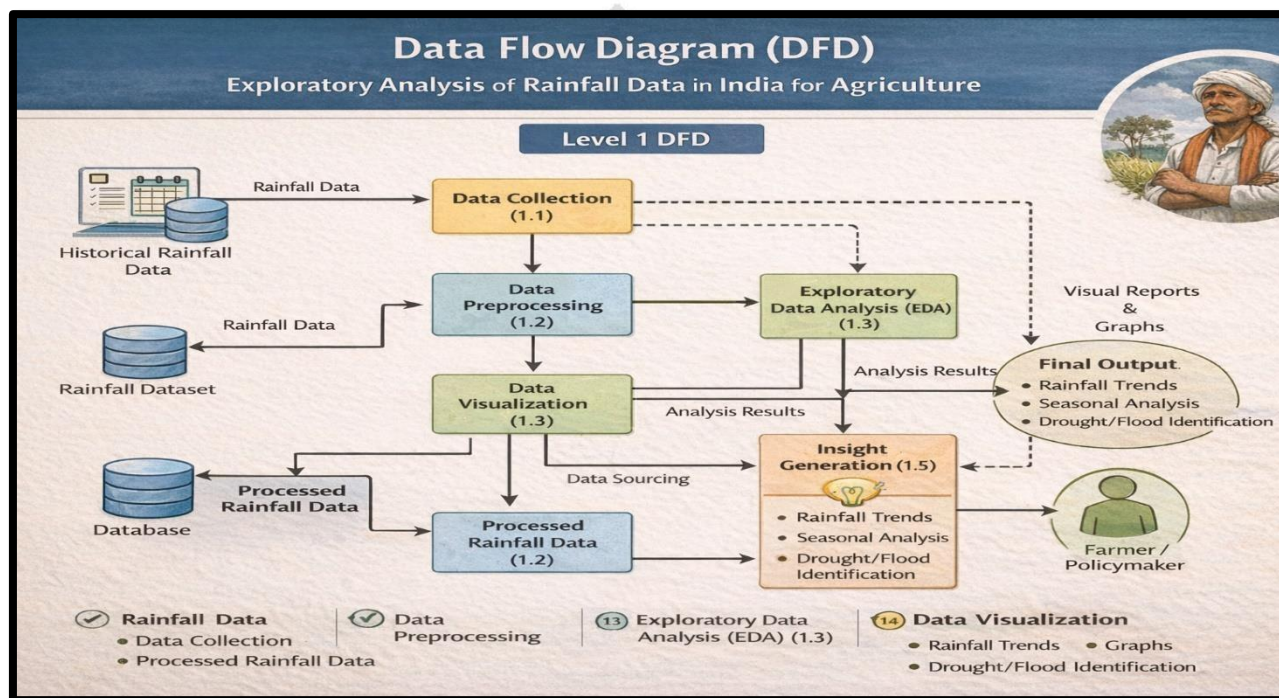
The system should run on a standard computer without requiring high-end infrastructure.



Exploratory Analysis of Rain Fall Data in India for Agriculture

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	3MARKS

3.3 - Data Flow Diagram



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMID89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	3 MARKS

3.4 Technology Stack

The Technology Stack refers to the set of tools, programming languages, libraries, and platforms used to develop and implement the rainfall data analysis system.

For this project, a Python-based data analysis environment is used to collect, preprocess, analyze, and visualize rainfall data efficiently.

Programming Language:

Python:

Python is used as the primary programming language for this project because:

- It is simple and easy to understand.
- It provides powerful libraries for data analysis.
- It supports statistical computation and visualization.
- It is widely used in data science and analytics.

Python enables efficient processing of large rainfall datasets.

Data Manipulation Libraries:

Pandas:

Pandas is used for:

- Loading rainfall datasets
- Handling missing values
- Data cleaning and preprocessing
- Organizing structured data
- Performing statistical summaries

It helps transform raw rainfall data into a structured format suitable for analysis.

NumPy

NumPy is used for:

- Numerical computations
- Mathematical operations

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Handling multi-dimensional arrays
- Performing statistical calculations

It supports efficient data processing and computation.

Data Visualization Tools

Matplotlib:

Matplotlib is used to:

- Create line charts for rainfall trends
- Generate bar charts for regional comparisons
- Visualize seasonal rainfall patterns

It provides basic and customizable plotting features.

Seaborn

Seaborn is used for:

- Advanced statistical visualizations
- Heatmaps for rainfall distribution
- Comparative graphical analysis

It enhances visual representation and improves readability.

Development Environment

Jupyter Notebook / IDE:

The project is developed using:

- Jupyter Notebook for interactive data analysis
or
- Python IDE (such as VS Code or PyCharm)

These environments allow easy coding, visualization, debugging, and documentation.

Dataset Source

The project uses:

- Historical rainfall datasets
- Region-wise or state-wise rainfall records
- Seasonal rainfall data

The dataset is stored locally and processed using Python libraries.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	5 MARKS

CHAPTER-04

PROBLEM CONTEXT:

India is predominantly an agriculture-based country, with a significant portion of its farming dependent on monsoon rainfall. The variability, unpredictability, and uneven spatial distribution of rainfall create multiple challenges:

- Uncertain crop planning due to irregular monsoon patterns
- Increased risk of droughts and floods
- Crop failure and reduced agricultural productivity
- Financial instability for farmers
- Lack of data-driven decision-making in rural farming systems

Although historical rainfall data is available through agencies such as the India Meteorological Department, it is often underutilized due to the absence of systematic exploratory analysis and interpretation tailored for agricultural decision-making.

CORE PROBLEM STATEMENT:

Farmers and agricultural planners lack a structured analytical framework that converts historical rainfall data into meaningful insights for crop selection, seasonal planning, irrigation management, and risk mitigation.

The gap lies not in data availability, but in data interpretation and actionable intelligence generation.

PROPOSED SOLUTION:

The project proposes an AI/ML-based Exploratory Data Analysis (EDA) framework that:

- Analyzes historical rainfall trends across Indian states and regions
- Identifies seasonal patterns (Southwest & Northeast monsoon variations)
- Detects anomalies such as extreme rainfall or drought years
- Examines long-term rainfall variability
- Correlates rainfall distribution with agricultural cycles

THE SOLUTION INTEGRATES:

- Data preprocessing techniques
- Statistical analysis
- Visualization models
- Trend identification methods
- Pattern recognition using machine learning

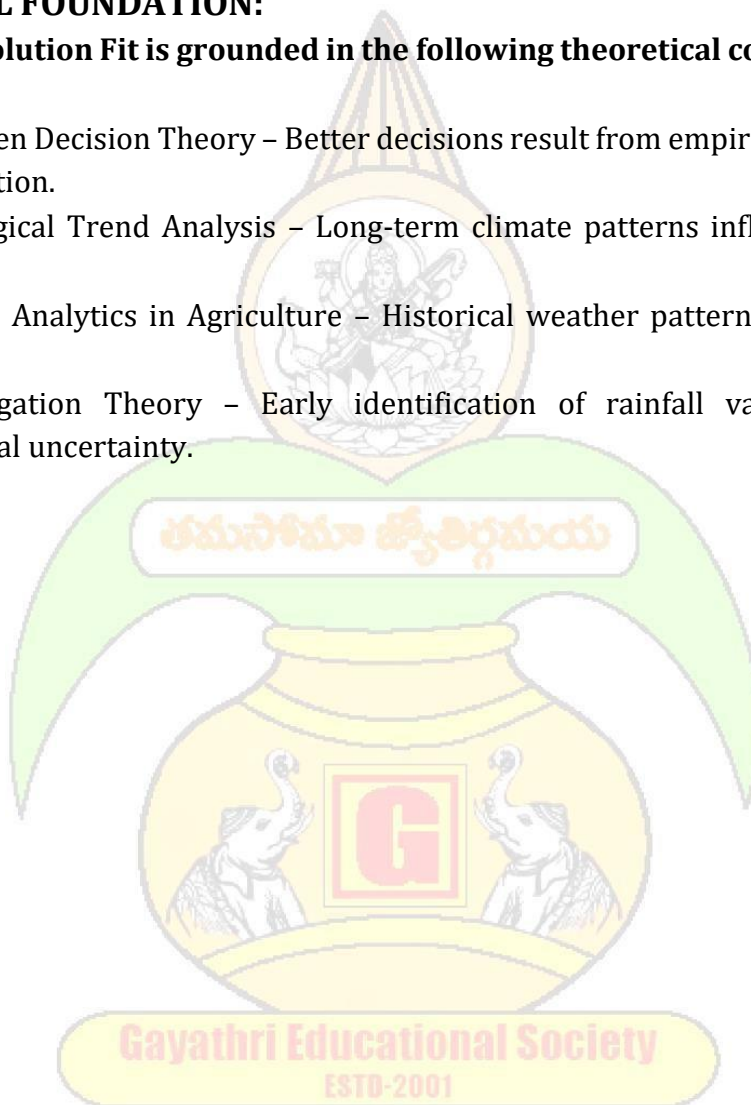
VALUE PROPOSITION THE PROJECT DELIVERS VALUE BY:

- Converting raw rainfall data into structured agricultural insights
- Supporting strategic crop planning
- Reducing economic risk for farmers
- Enhancing sustainable agricultural practices
- Assisting policymakers in rainfall-dependent resource allocation

THEORETICAL FOUNDATION:

The Problem–Solution Fit is grounded in the following theoretical concepts:

- Data-Driven Decision Theory – Better decisions result from empirical analysis rather than intuition.
- Climatological Trend Analysis – Long-term climate patterns influence agricultural cycles.
- Predictive Analytics in Agriculture – Historical weather patterns can guide future planning.
- Risk Mitigation Theory – Early identification of rainfall variability reduces agricultural uncertainty.



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	5MARKS

4.2 PROPOSED SOLUTION PROPOSED SOLUTION:

The proposed solution involves designing and implementing a comprehensive AI/ML-based Exploratory Data Analysis (EDA) framework that systematically studies historical rainfall data across different regions of India to enhance agricultural planning and sustainability. India's agriculture is heavily dependent on monsoon rainfall, and variations in seasonal precipitation significantly impact crop yield, water availability, and rural livelihoods. To address this challenge, the system collects long-term rainfall datasets from authoritative sources such as the India Meteorological Department and other verified meteorological repositories. These datasets typically include monthly, seasonal, and annual rainfall records categorized by states and districts.

The solution begins with data preprocessing, where missing values, inconsistencies, and outliers are handled using statistical imputation and normalization techniques. The cleaned dataset is then subjected to descriptive statistical analysis to understand central tendencies, variability, and distribution patterns of rainfall across different regions. Seasonal decomposition techniques are applied to distinguish long-term trends from cyclical monsoon patterns, particularly focusing on the Southwest and Northeast monsoon systems that govern India's agricultural cycle.

Further, machine learning algorithms are incorporated to enhance pattern recognition and anomaly detection. The system identifies drought-prone and flood-prone years by analyzing deviations from long-term rainfall averages. Clustering techniques may be used to group regions with similar rainfall behavior, enabling region-specific agricultural recommendations. Correlation analysis is also performed to understand the relationship between rainfall distribution and crop-growing seasons, thereby supporting strategic crop selection and sowing time decisions.

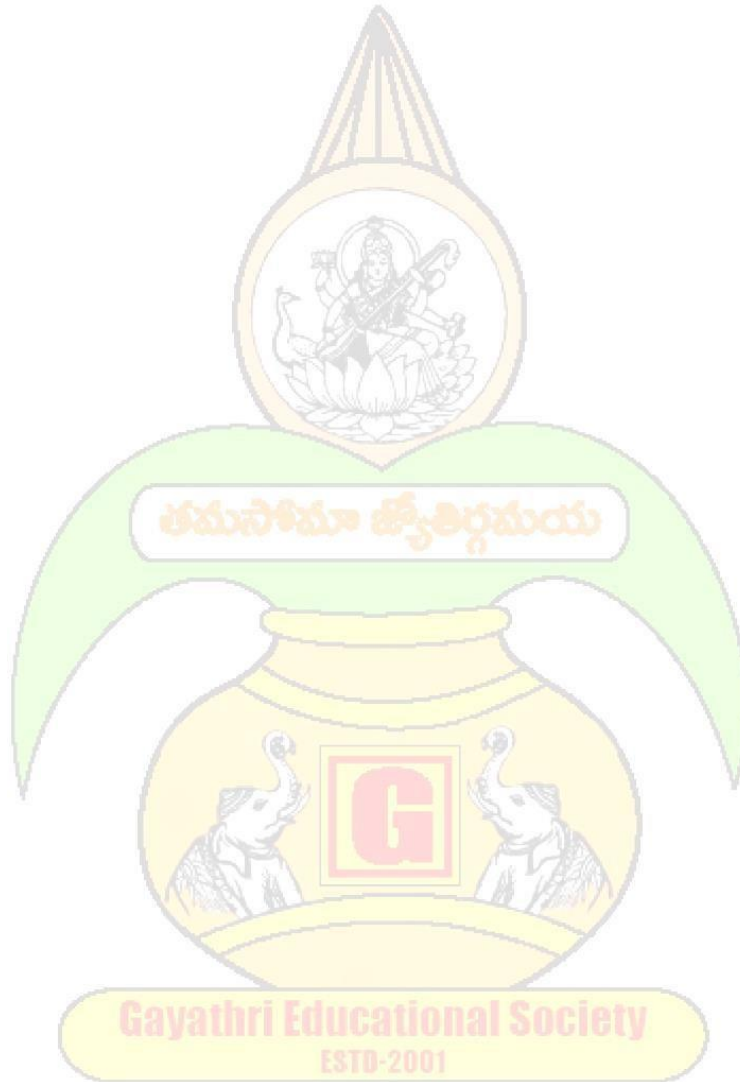
Visualization plays a critical role in the proposed solution. Interactive charts, heatmaps, trend graphs, and seasonal comparison plots are generated to make complex rainfall patterns easily interpretable for stakeholders. These visual representations transform raw meteorological data into actionable insights, allowing farmers, agricultural planners, and policymakers to make informed decisions based on empirical evidence rather than assumptions.

Additionally, the framework is designed to be scalable and adaptable for future integration with predictive models. While the primary focus is exploratory analysis, the structured insights generated by the system can later be extended into rainfall forecasting

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

or yield prediction models. This ensures long-term relevance and adaptability of the project.

Overall, the proposed solution bridges the gap between raw climate data and practical agricultural application by providing a structured, analytical, and data-driven approach. It empowers stakeholders to mitigate risks associated with rainfall variability, optimize resource allocation, improve irrigation planning, and promote sustainable agricultural development across India.

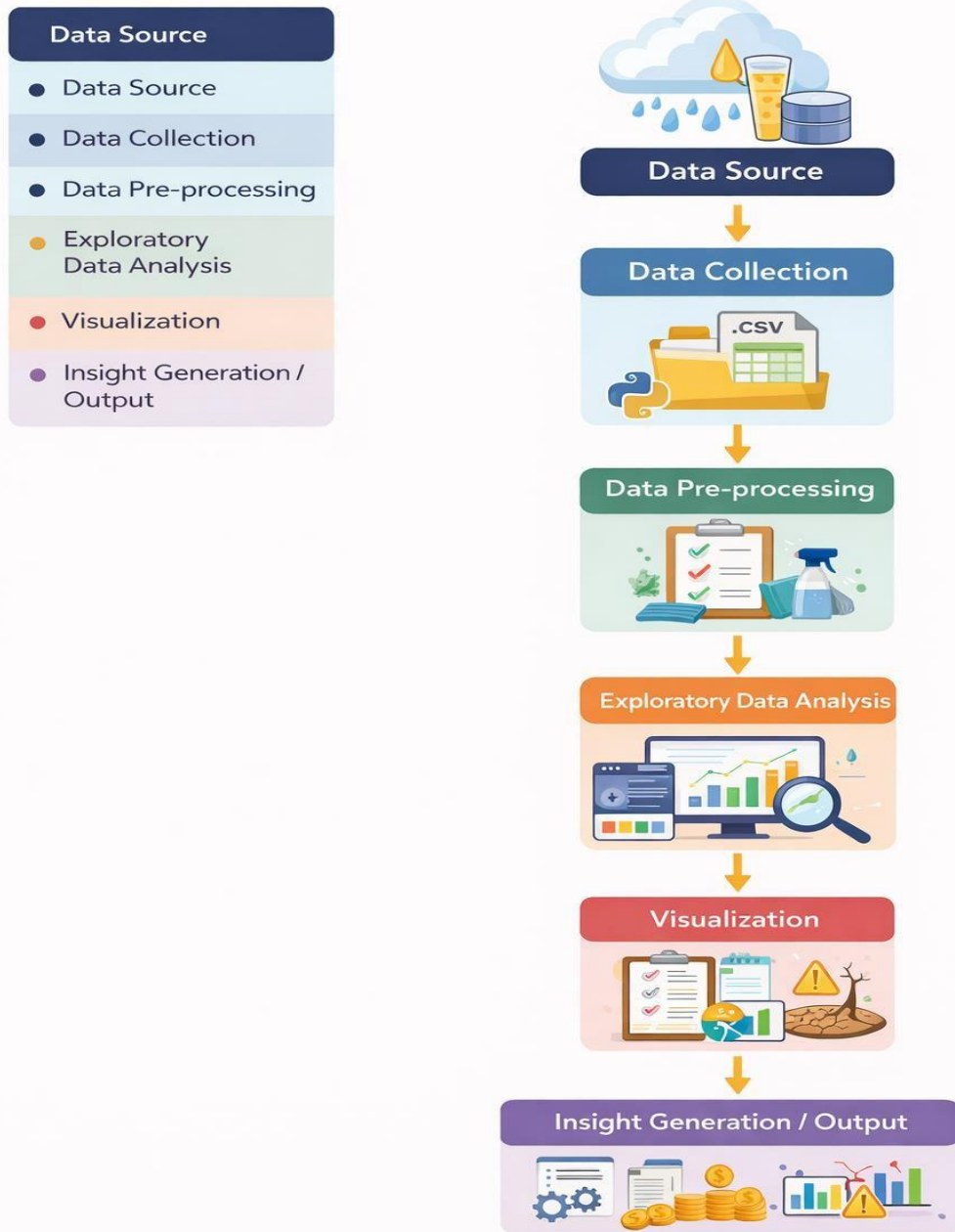


EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	5 MARKS

4.3 – Solution Architecture:

Exploratory Analysis of Rainfall Data in India for Agriculture



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	1 MARKS

Chapter – 5

Project Planning & Scheduling

5.1 - Project Milestones & Tasks

1 Data Collection

Objective: Gather high-quality, relevant data required for the project.

Tasks:

- Identify reliable data sources (databases, APIs, web scraping, surveys, etc.)
- Collect structured and unstructured data
- Ensure data relevance and completeness
- Store data in a centralized database or storage system
- Maintain data documentation for reference

Deliverables:

- Raw dataset
- Data source documentation
- Data storage setup

2 Data Pre-Processing

Objective: Clean and prepare the data for model training and analysis.

Tasks:

- Remove duplicates and handle missing values
- Handle outliers and inconsistent data
- Data normalization or scaling
- Feature engineering and selection
- Encode categorical variables
- Split dataset into training and testing sets

Deliverables:

- Cleaned dataset

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Feature-engineered dataset
- Preprocessing scripts

Model Building

Objective: Develop and train a machine learning model.

Tasks:

- Select appropriate algorithm(s)
- Train model using training dataset
- Hyperparameter tuning
- Model validation and evaluation
- Compare performance metrics
- Finalize best-performing model

Deliverables:

- Trained model
- Evaluation report (Accuracy, Precision, Recall, F1-score, etc.)
- Saved model file

API Integration

Objective: Expose the trained model via an API for external access.

Tasks:

- Develop REST API endpoints
- Integrate model with backend framework (Flask / FastAPI / Django)
- Implement request validation
- Add error handling
- Test API responses
- Secure API (authentication if required)

Deliverables:

- Functional API
- API documentation (Swagger/Postman collection)
- Deployment-ready backend

Web Integration

Objective: Integrate API into a web-based user interface.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Tasks:

- Design user-friendly frontend interface
- Connect frontend to backend API
- Display model predictions dynamically
- Implement input validation
- Perform end-to-end testing
- Deploy web application

Deliverables:

- Fully functional web application
- User interface design
- Live deployment link (if applicable)



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	2 MARKS

5.2 - Sprint Delivery Plan

◆ Phase 1: Live Sessions (Week 1–6)

Objective: Build strong foundational knowledge and prepare interns for real-time project development.

■ Sprint 1 (Week 1–2): Fundamentals

& Tools Focus Areas:

- Introduction to Internship Program
- Programming Fundamentals (Python / Relevant Tech Stack)
- Git & GitHub
- Development Environment Setup
- Basics of Databases

Deliverables:

- Setup development environment
- GitHub repository creation
- Mini practice assignments

■ Sprint 2 (Week 3–4): Data & Backend

Foundations Focus Areas:

- Data Handling (Pandas / Data Structures)
- Data Cleaning Techniques
- Introduction to APIs
- Backend Basics (Flask / FastAPI)
- SQL & Database Integration

Deliverables:

- Data preprocessing assignment
- Basic API development task

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Database connectivity demo

Sprint 3 (Week 5–6): Machine Learning & Deployment

Basics Focus Areas:

- Machine Learning Fundamentals
- Model Training & Evaluation
- REST API Integration with Model
- Introduction to Web Integration
- Deployment Overview

Deliverables:

- Simple ML Model
- Model evaluation report
- API with working prediction endpoint

◆ Phase 2: Project Work (Week 7–15)

Objective: Apply learned skills to build a complete end-to-end project.

Sprint 4 (Week 7–8): Project Planning & Data

Collection Activities:

- Finalize project topic
- Define problem statement
- Collect dataset
- Perform initial data analysis (EDA)

Deliverables:

- Project proposal document
- Dataset documentation
- EDA report

Sprint 5 (Week 9–10): Data Preprocessing & Feature

Engineering Activities:

- Clean dataset
- Handle missing values & outliers

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Feature engineering
- Data transformation
- Train-test split

Deliverables:

- Cleaned dataset
- Preprocessing pipeline
- Feature documentation

Sprint 6 (Week 11–12): Model Development &

Optimization Activities:

- Train multiple models
- Hyperparameter tuning
- Model comparison
- Performance evaluation

Deliverables:

- Best performing model
- Evaluation metrics report
- Saved model artifact

Sprint 7 (Week 13–14): API Development &

Integration Activities:

- Develop REST API
- Integrate trained model
- Implement validation & error handling
- Test API endpoints

Deliverables:

- Functional API
- API documentation
- Backend deployment

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE



Sprint 8 (Week 15): Web Integration & Final

Deployment Activities:

- Develop frontend interface
- Connect frontend with API
- End-to-end testing
- Deployment & final presentation

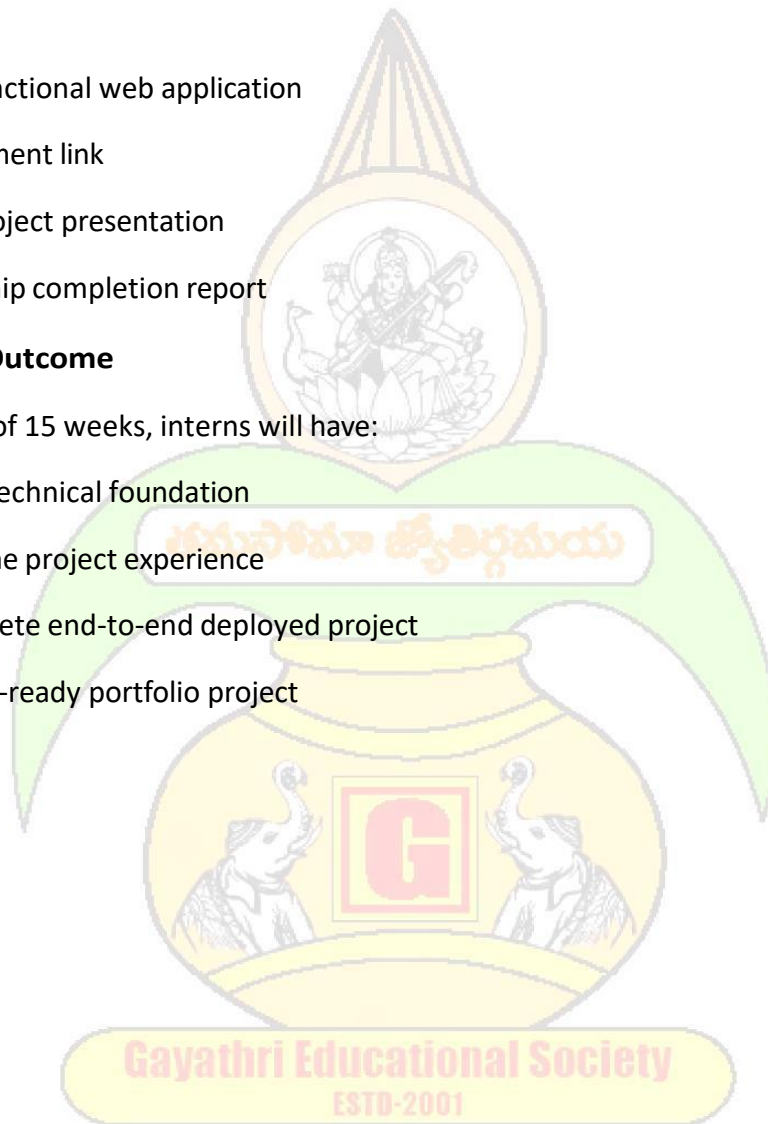
Deliverables:

- Fully functional web application
- Deployment link
- Final project presentation
- Internship completion report

Final Outcome

By the end of 15 weeks, interns will have:

- Strong technical foundation
- Real-time project experience
- A complete end-to-end deployed project
- Industry-ready portfolio project



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	1MARKS

5.3 - Project Progress Tracking

1. Zoho Cliq Workspace Structure

◆ Channels Setup

Create the following channels for organized communication:

🔗 1. Announcements

- Official updates
- Sprint start/end notifications
- Deadlines & evaluation updates
- Meeting schedules

🔗 2. Project-discussion

- Technical queries
- Implementation discussions
- Code review discussions
- Issue troubleshooting

🔗 3. Daily-updates

- Daily progress reports
- Blockers
- Completed tasks

🔗 4. Resources

- Shared datasets
- Documentation links
- Recorded session links
- API documentation

🔗 5. Team-specific-channels (if multiple teams)

Example:

- #team-alpha
- #team-beta

2. Sprint-Based Tracking Method

Each sprint will follow a structured reporting cycle.

● Daily Progress Update Format (Posted in #daily-updates)

Every intern must post:

Format:

Date:

Sprint:

Tasks Completed:

Tasks In Progress:

Blockers (if any):

Plan for Tomorrow:

● Weekly Sprint Review (Every Weekend)

Mentor will post:

- Sprint Goals
- Completed Milestones
- Pending Tasks
- Risk Areas
- Next Week Targets

3. Task Tracking System

Option 1: Zoho Cliq Tasks Feature

Use built-in task management in Zoho Cliq:

- Assign tasks to interns
- Set deadlines
- Track completion status
- Add task priority (High / Medium / Low)

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Task Status Workflow:

- To Do
- In Progress
- Completed
- Blocked

Option 2: Zoho Projects Integration (Optional)

For advanced tracking, integrate with:

- Zoho Projects

Use:

- Kanban Board
- Gantt Chart
- Milestone tracking
- Automated reminders

4. Sprint Progress Monitoring Dashboard

Track the following metrics weekly:

- % Tasks Completed
- API Development Progress
- Model Accuracy Improvement
- Deployment Readiness
- Bug Count
- Attendance in Live Sessions

Mentor shares a weekly progress summary in:

🔔 #announcements channel

5. Milestone Tracking Structure

Milestone	Week	Status	Owner	Remarks
Data Collection	Week 7-8	☐	Team	
Data Preprocessing	Week 9-10	☐	Team	
Model Building	Week 11-12	☐	Team	

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

API Integration	Week 13-14	☐	Team	
Web Integration	Week 15	☐	Team	

6. Escalation Process

If blocker > 24 hours:

- Post in #project-discussion
- Tag mentor
- If unresolved → Schedule quick call via Zoho Cliq
- Update resolution summary in channel

7. Performance Evaluation Criteria

Evaluation will be based on:

- Daily update consistency
- Sprint milestone completion
- Code quality
- Participation in discussions
- Final project delivery
- Timely submissions

Weekly Workflow Summary

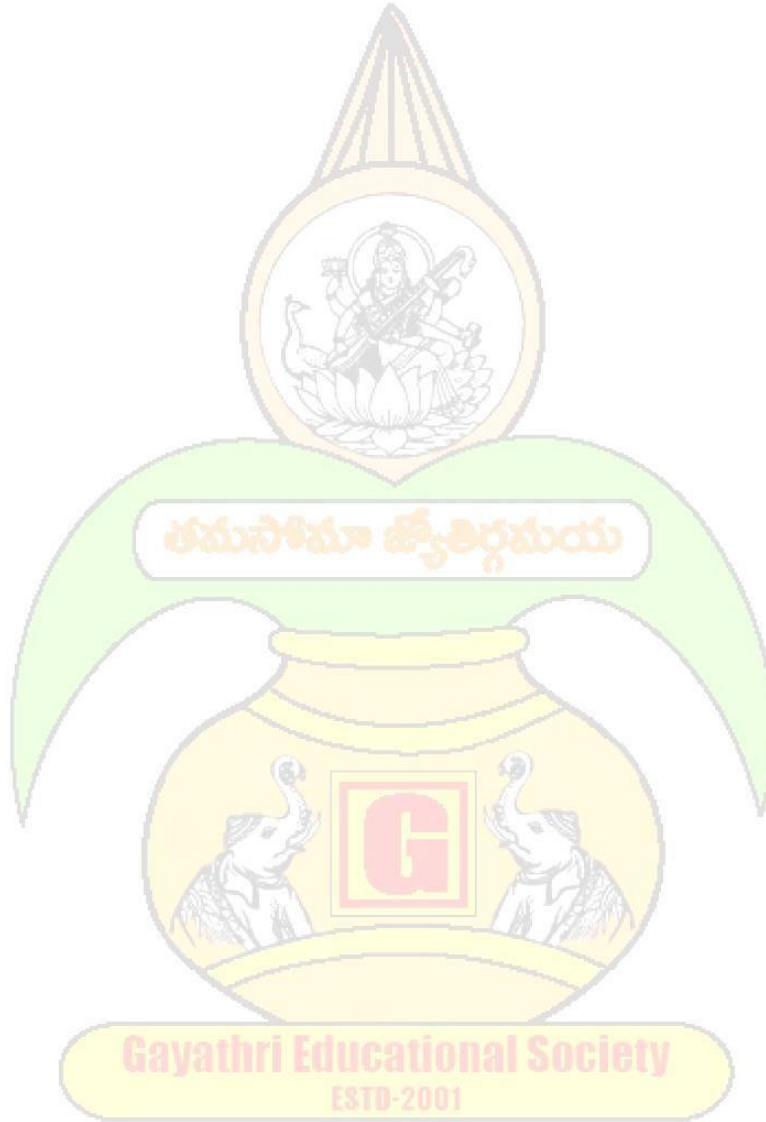
Day	Activity
Monday	Sprint planning post
Tue-Thu	Development & daily updates
Friday	Progress review
Saturday	Sprint demo
Sunday	Feedback & planning

Final Outcome

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Using Zoho Cliq ensures:

- Clear communication
- Accountability
- Structured sprint tracking
- Real-time problem resolution
- Professional workflow management



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	1 MARKS

5.4 - Team Management Tools for Agile Planning

✧ What is Jira?

Jira is an Agile project management and issue-tracking tool developed by Atlassian. It supports Scrum and Kanban methodologies, helping teams plan, track, and release software efficiently.

📁 Jira Project Structure for Internship

1 Project Creation

Create a project with:

- Project Name: Internship Capstone Project
- Template: Scrum (Recommended)
- Project Type: Software Development

🔧 Issue Types Configuration

Define standard issue types:

- ● Epic – Major project phases
- ● Story – Feature or functionality
- ● Task – Smaller implementation steps
- ● Bug – Errors or defects
- ● Sub-task – Breakdown of tasks

📌 Suggested Epics (Based on Your Milestones)

Epic	Description
Data Collection	Dataset gathering & validation
Data Preprocessing	Cleaning & feature engineering
Model Development	ML training & evaluation

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

API Integration	Backend & model API
Web Integration	Frontend & deployment

Sprint Planning Structure

◆ Sprint Duration

- 2 Weeks per Sprint
- Total: 4–5 Sprints (Project Phase)

◆ Sprint Workflow

1. Create Sprint in Backlog
2. Add Stories/Tasks to Sprint
3. Assign Tasks to Interns
4. Set Priority (High/Medium/Low)
5. Estimate Story Points
6. Start Sprint

Workflow Status Configuration

Customize workflow:

-  To Do
-  In Progress
-  In Review
-  Done
-  Blocked

This ensures transparent tracking of task movement.

Agile Boards in Jira

Scrum Board

Used for:

- Sprint planning
- Daily standups

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Tracking sprint progress

Kanban Board (Optional)

Used for:

- Continuous workflow
- API & bug tracking

Agile Reports for Monitoring

Jira provides built-in reports:

-  Burndown Chart – Sprint progress tracking
-  Velocity Chart – Team performance
-  Sprint Report – Completed vs pending work
-  Bug Report – Defect tracking

Mentors can review reports weekly to evaluate progress.

Role-Based Access

Define permissions:

Role	Responsibilities
Project Admin	Configure board & workflow
Scrum Master	Sprint planning & review
Developer (Intern)	Task implementation
Reviewer	Code & feature review

Daily Standup Format (Using Jira Board)

Each intern updates:

- What was completed yesterday
- What will be done today
- Any blockers

Tasks must be moved across workflow stages accordingly.

Integration Capabilities

Jira can integrate with:

- GitHub (code tracking)
- CI/CD tools
- Slack or Zoho Cliq (notifications)
- Confluence (documentation)

| ^a Example Sprint Breakdown (2 Weeks)

Sprint Goal: Complete Data Preprocessing

Planned Stories:

- Clean missing values
- Outlier detection
- Feature scaling
- Train-test split

Expected Outcome:

Clean dataset ready for model training.

✓ Benefits of Using Jira

- Structured Agile Planning
- Clear Accountability
- Transparent Sprint Tracking
- Performance Analytics
- Real-time Status Visibility
- Industry-standard project management tool

🏁 Final Outcome

By implementing Jira for Agile planning, the internship project will follow:

- Scrum-based sprint execution
- Organized backlog management
- Efficient task delegation
- Measurable progress tracking
- Professional software development workflow

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	2MARKS

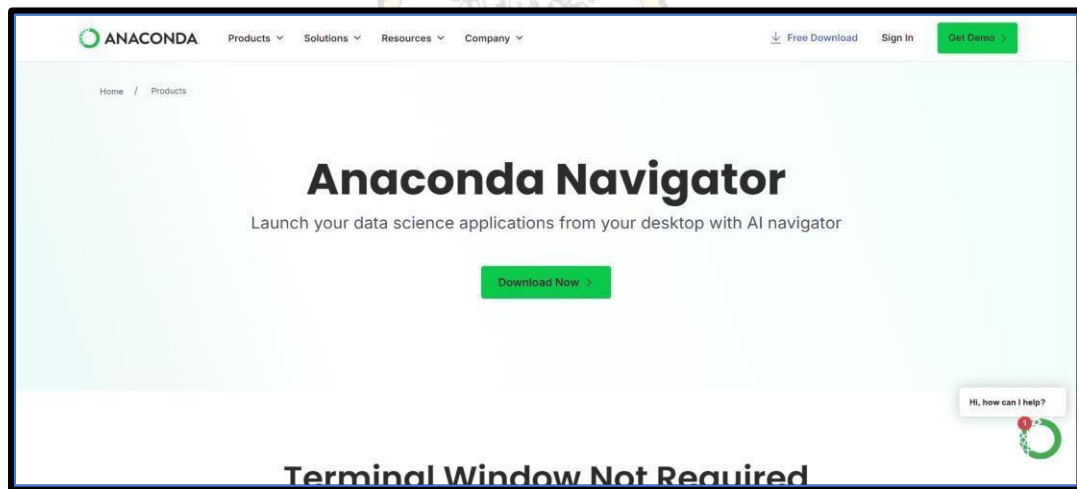
CHAPTER-6 PROJECT DEVELOPMENT

6.1-PRE-REQUISITES:

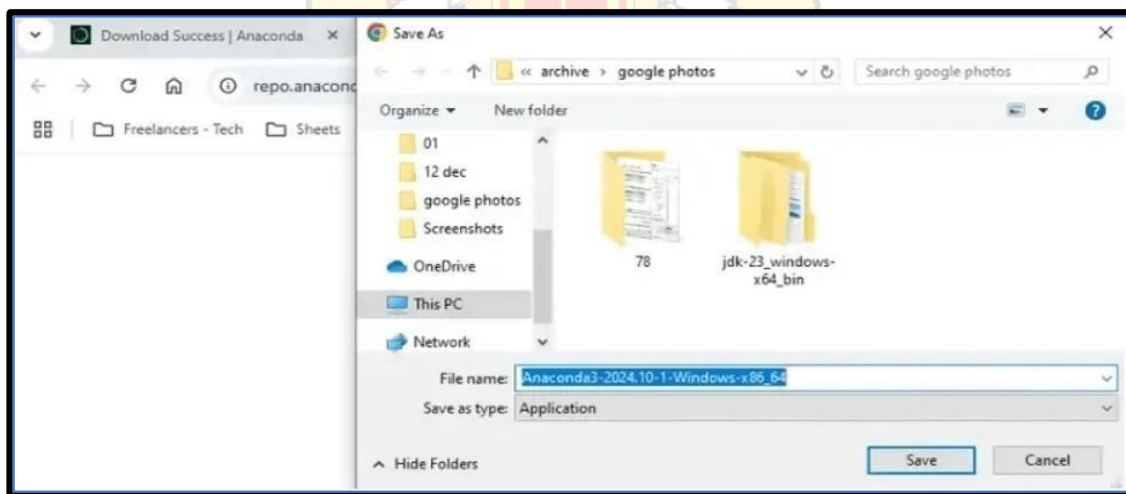
In order to develop this project, we need to install the following software/packages:

Step 1: Anaconda Navigator

- Open this link <https://www.anaconda.com/products/navigator>

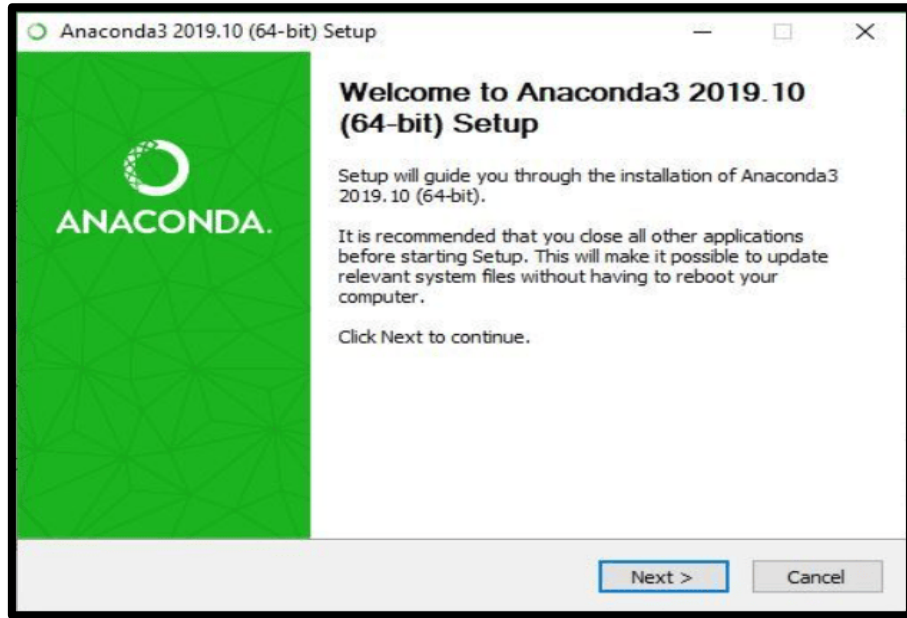


- Click on Download button

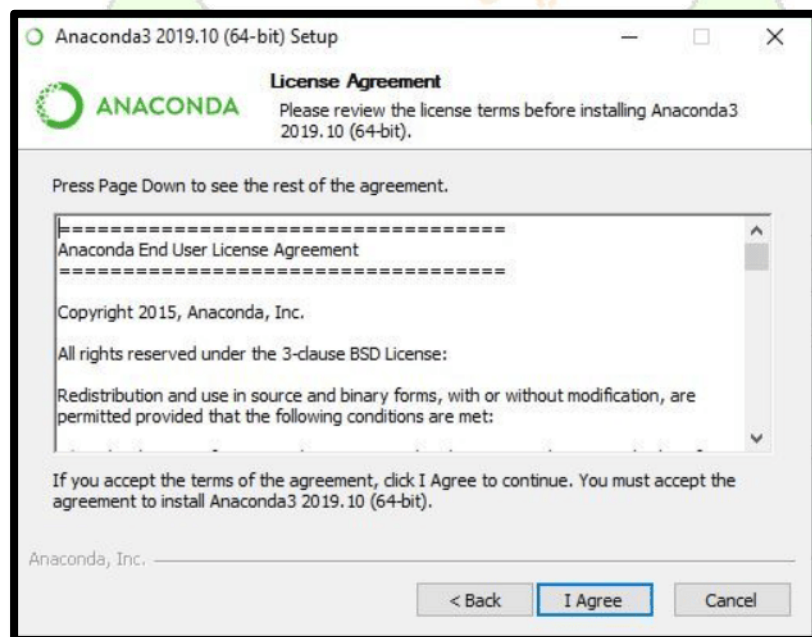


EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- After downloading the Anaconda Navigator double click on the file
- Select the Windows Installer
- Begin the installation process.

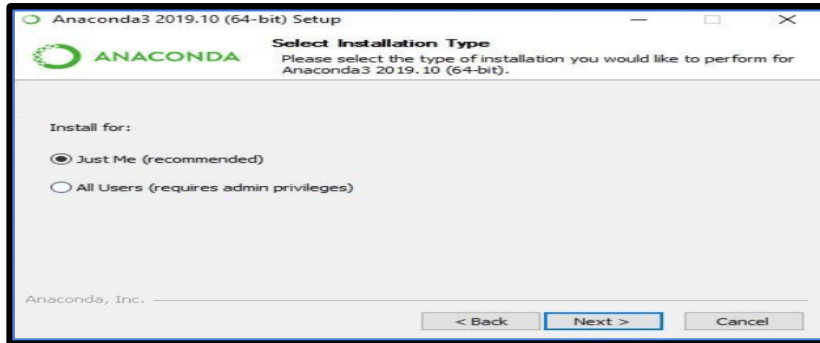


- Getting through license agreement

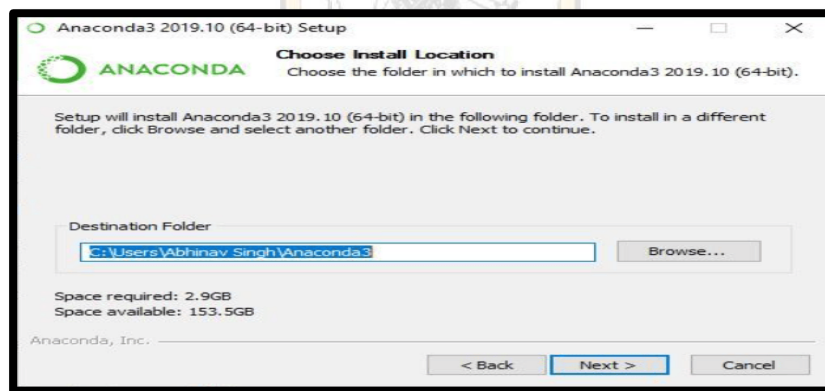


EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

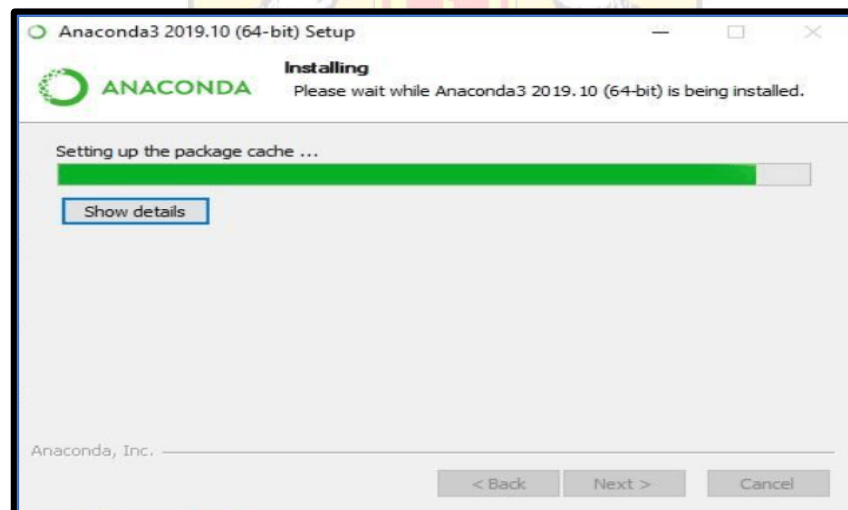
- Select Installation Type



- Select the path where you wish to install the file extract or and click "Next" to proceed ahead.
- Advance installation options

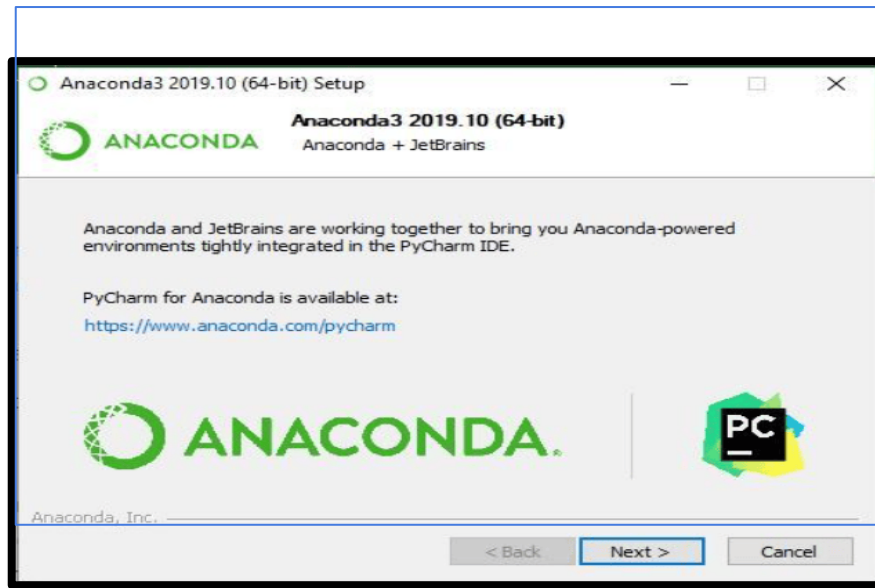


- Click Install to start the Anaconda Installation process.

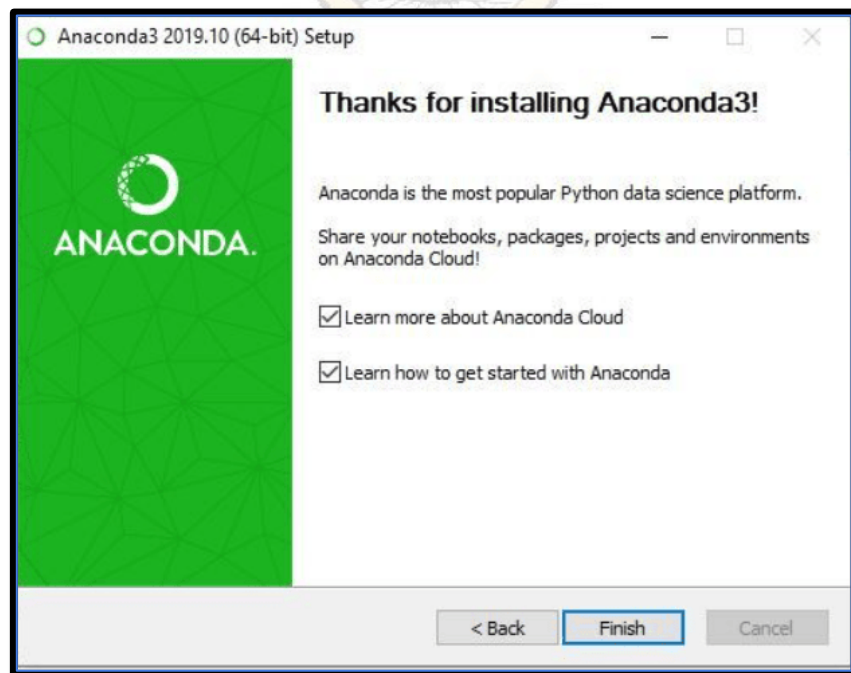


EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Recommendation to install PyCharm

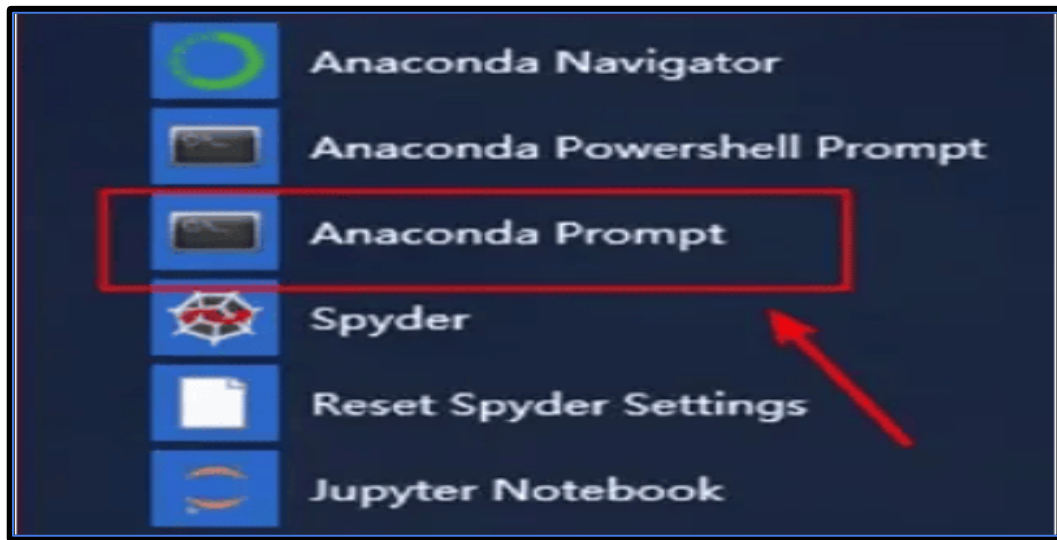


- Once the installation gets complete, Click Finish to complete the process.



- Click on the Start Menu and search for "Anaconda Prompt" and click to open it.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE



- Run the Program to check for the Anaconda Version

Type the following command to check for the installed version of conda:

Conda -version

```
Anaconda Prompt

(base) C:\Users\SHREYA>conda --version
conda 23.3.1

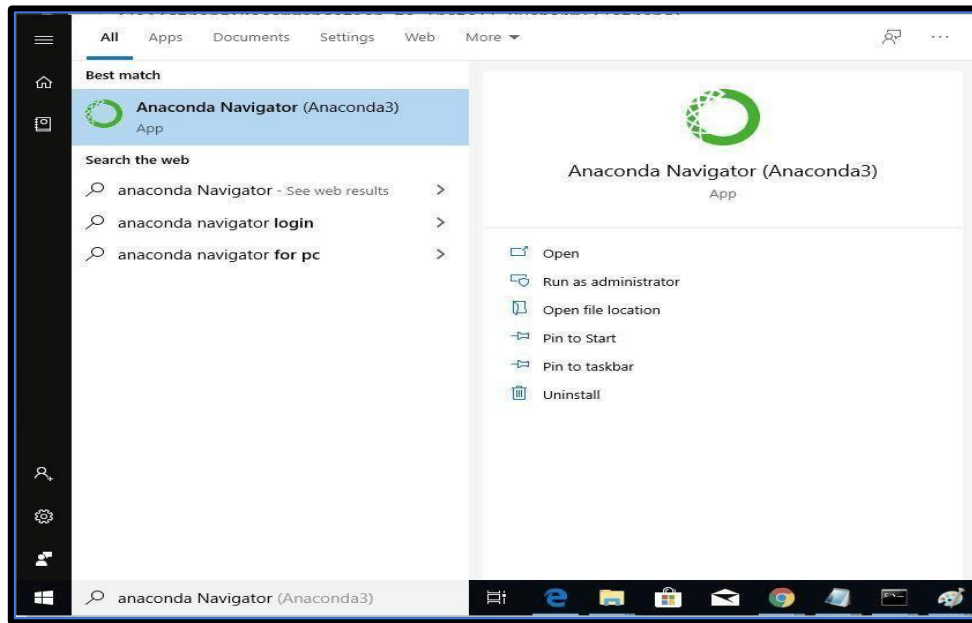
(base) C:\Users\SHREYA>python --version
Python 3.10.9

(base) C:\Users\SHREYA>
```

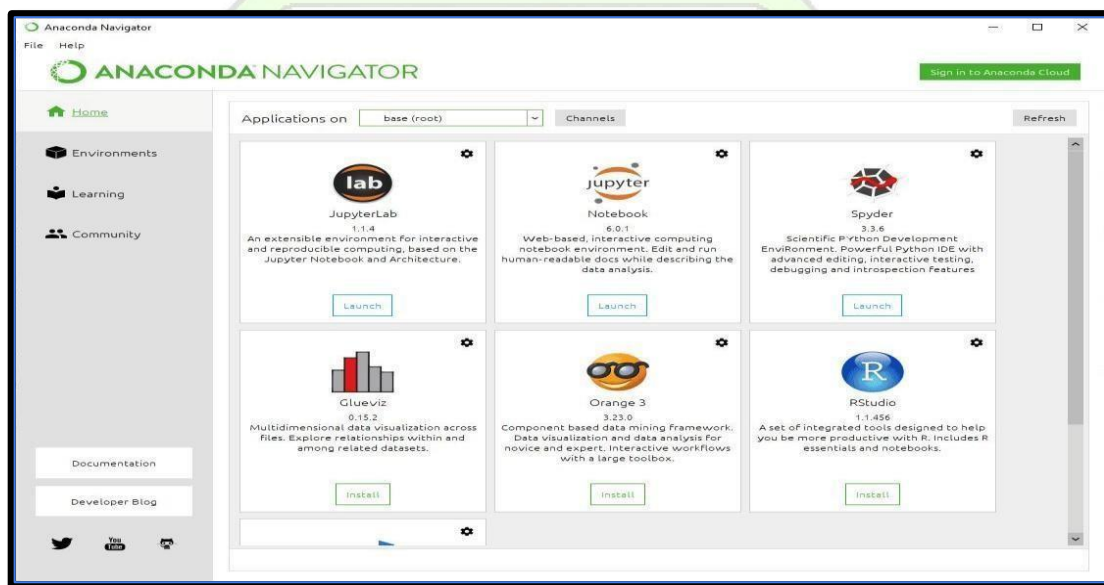
- Access Anaconda Navigator
- Once the installation process is done, Anaconda can be used to perform multiple operations. To begin using Anaconda, search for Anaconda Navigator from the

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Start Menu in Windows PC.



➤ Explore Navigator & Features:



How to Install PyCharm:

Step1: Download PyCharm

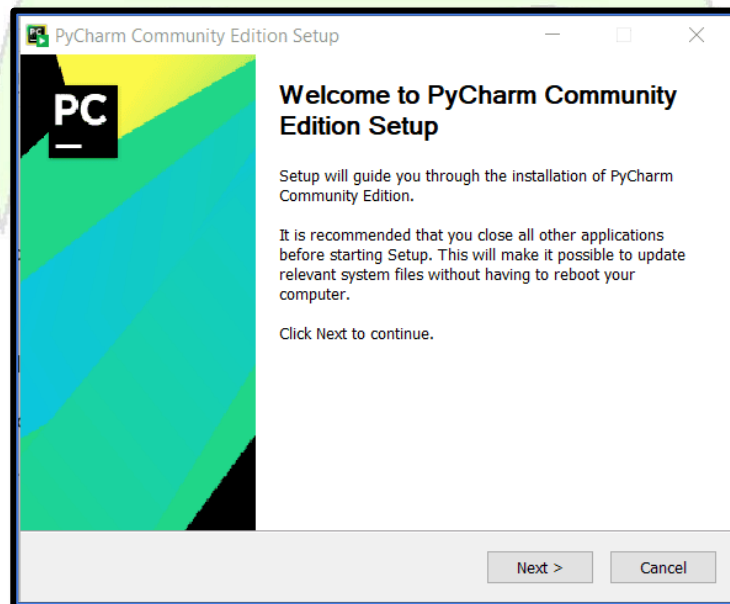
- Go to the Jet Brains PyCharm download page
- Choose either the Community edition(free) or the Professional edition(paid).



- Download the installer according to your preference.

Step - 2: Run the Installer

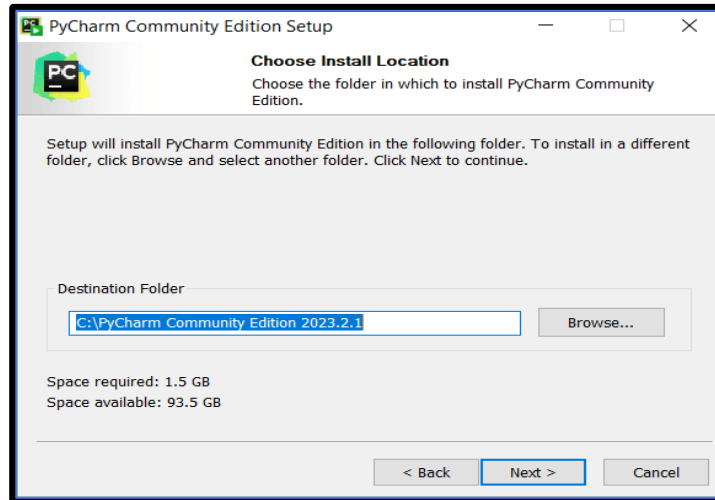
- Locate the downloaded installer file and double-click it to run.
- Follow the on-screen instructions.



Step-3: Choose Installation Location

- Select the folder where you want to install PyCharm.

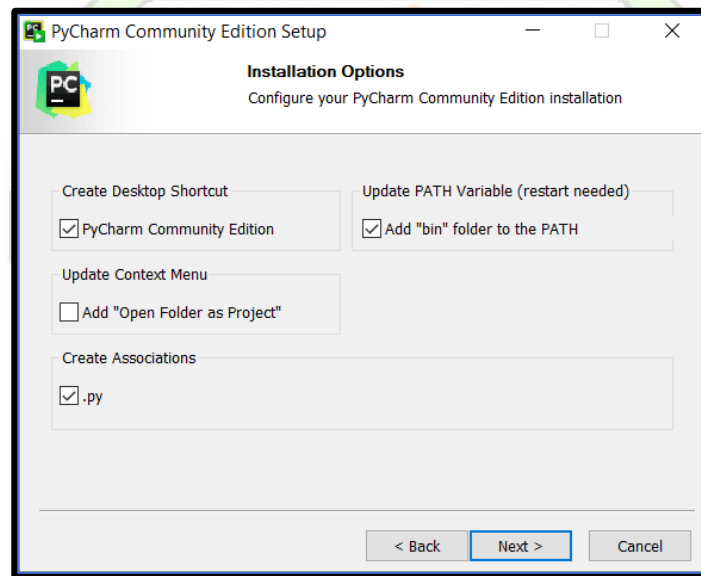
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE



Step-4: Select Installation Options

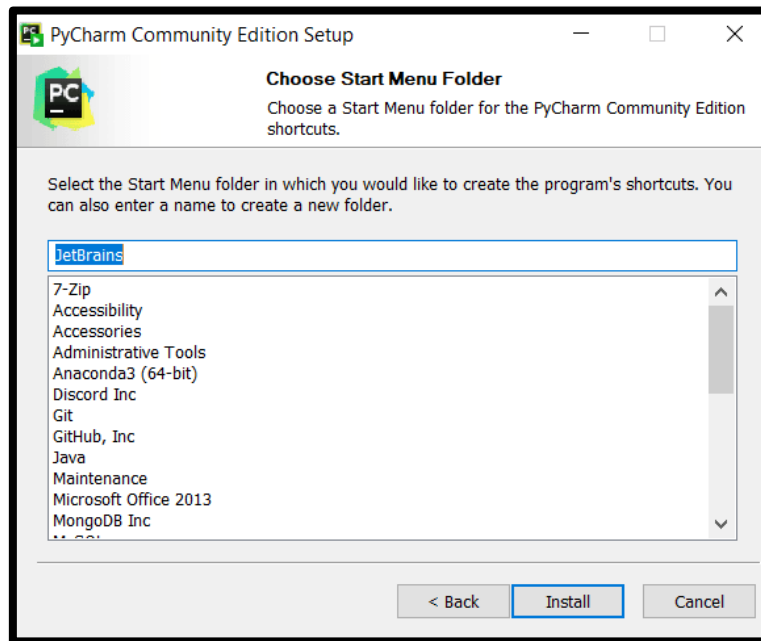
You can select the following options according to your preference:

- Create Desktop Shortcut-adds a shortcut on your desktop.
- Add PyCharm to PATH-allows you to run PyCharm from the command line.
- Associate.py files with PyCharm-makes PyCharm the default editor for Python files.

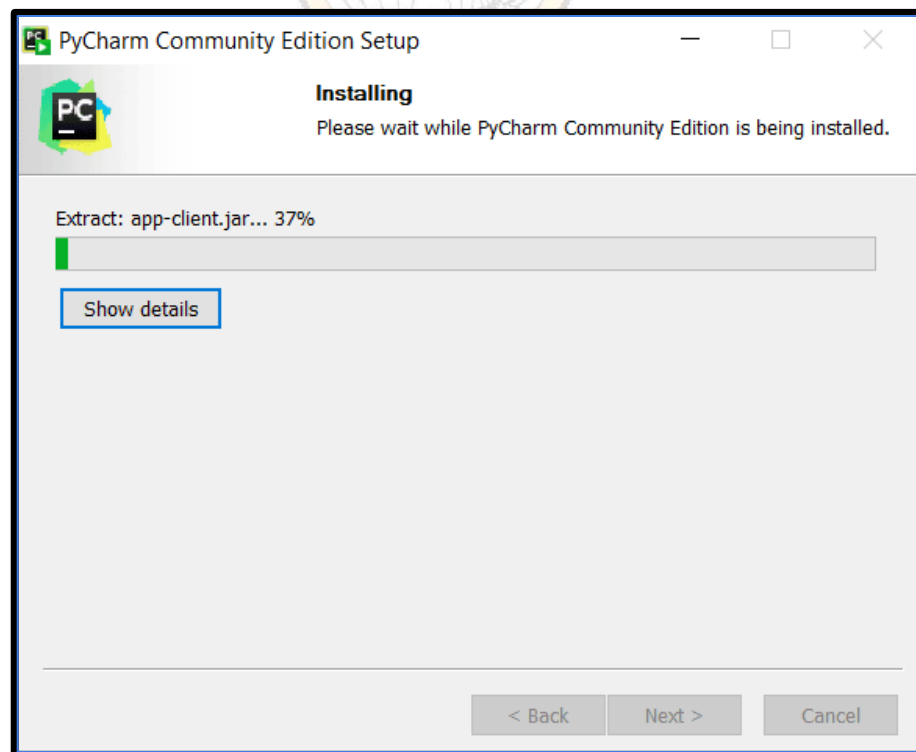


- Choose folder name where your project will be saved as default.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE



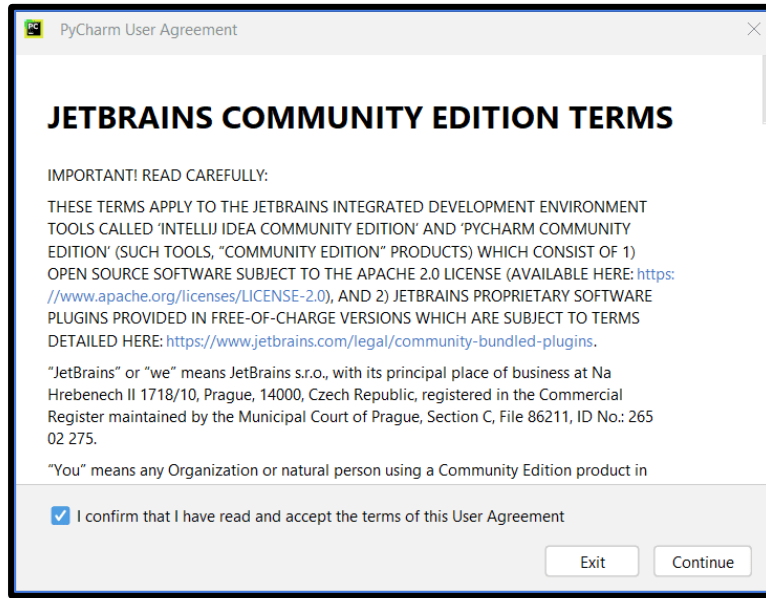
- Now it will download the PyCharm



Step5: Complete Installation

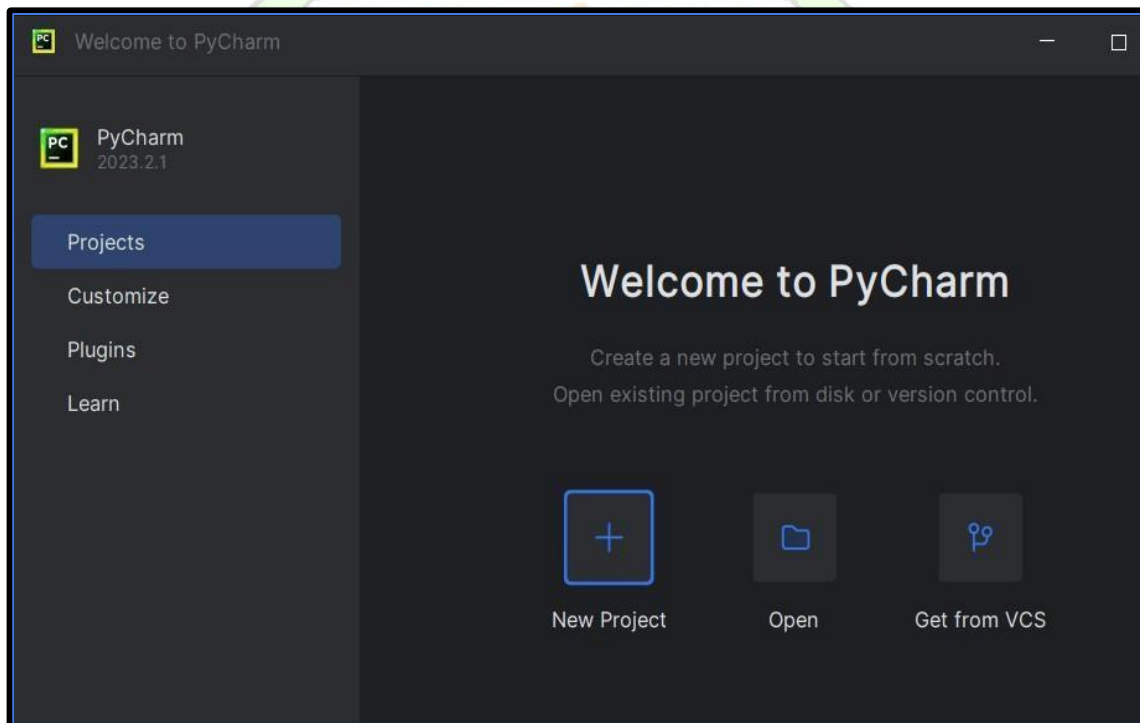
- Accept the license agreement and continue.
- PyCharm will be installed.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE



Step6: Launch PyCharm

- Once installed, launch PyCharm either from the Start menu or the desktop shortcut.



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

How to install Packages:

Now you are using PyCharm IDE, follow the below steps to download the required packages:

Type “pip install NumPy” and click enter.

```
(.venv) PS D:\LT Python Project> pip install numpy
Requirement already satisfied: numpy in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (2.2.3)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\LT Python Project> |
```

Like this we have to install all the packages

- Type “pip install pandas” and click enter

```
(.venv) PS D:\LT Python Project> pip install pandas
Requirement already satisfied: pandas in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (2.2.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.2.3)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.1)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\LT Python Project>
```

- Type “pip install scikit-learn” and click enter.

```
(.venv) PS D:\LT Python Project> pip install scikit-learn
Requirement already satisfied: scikit-learn in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (1.7.2)
Requirement already satisfied: numpy>=1.22.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (2.2.3)
Requirement already satisfied: scipy>=1.8.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (1.16.2)
Requirement already satisfied: joblib>=1.2.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (3.6.0)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\LT Python Project> |
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Type "pip install matplotlib" and click enter.

```
(.venv) PS D:\LT Python Project> pip install matplotlib
Requirement already satisfied: matplotlib in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (2.2.3)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\LT Python Project>
```

- Type "pip install SciPy" and click enter.

```
(.venv) PS C:\LT project> pip install scipy
Requirement already satisfied: scipy in c:\lt project\.venv\lib\site-packages (1.17.0)
Requirement already satisfied: numpy<2.7, >=1.26.4 in c:\lt project\.venv\lib\site-packages (from scipy) (2.4.2)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Type "pip install pickle-mixin" and click enter.

```
(.venv) PS C:\LT project> pip install pickle-mixin;
Requirement already satisfied: pickle-mixin in c:\lt project\.venv\lib\site-packages (1.0.2)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) PS C:\LT project>
```

- Type "pip install Flask" and click enter.

```
(.venv) PS C:\LT project> pip install Flask;
Requirement already satisfied: Flask in c:\lt project\.venv\lib\site-packages (3.1.2)
Requirement already satisfied: blinker>=1.9.0 in c:\lt project\.venv\lib\site-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in c:\lt project\.venv\lib\site-packages (from Flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in c:\lt project\.venv\lib\site-packages (from Flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in c:\lt project\.venv\lib\site-packages (from Flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in c:\lt project\.venv\lib\site-packages (from Flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in c:\lt project\.venv\lib\site-packages (from Flask) (3.1.5)
Requirement already satisfied: colorama in c:\lt project\.venv\lib\site-packages (from click>=8.1.3->Flask) (0.4.6)

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) PS C:\LT project>
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- Type” **pip install seaborn**” and click enter.

```
(.venv) PS C:\lt project> pip install seaborn;
Requirement already satisfied: seaborn in c:\lt project\.venv\lib\site-packages (0.13.2)
Requirement already satisfied: numpy<1.24.0,>=1.20 in c:\lt project\.venv\lib\site-packages (from seaborn) (2.4.2)
Requirement already satisfied: pandas>=1.2 in c:\lt project\.venv\lib\site-packages (from seaborn) (3.8.0)
Requirement already satisfied: matplotlib<3.6.1,>=3.4 in c:\lt project\.venv\lib\site-packages (from seaborn) (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (12.1.0)
Requirement already satisfied: pyparsing>=3 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\lt project\.venv\lib\site-packages (from matplotlib<3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\lt project\.venv\lib\site-packages (from pandas>=1.2->seaborn) (2025.3)
Requirement already satisfied: six>=1.5 in c:\lt project\.venv\lib\site-packages (from python-dateutil>=2.7->matplotlib<3.6.1,>=3.4->seaborn) (1.17.0)
[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	2MARKS

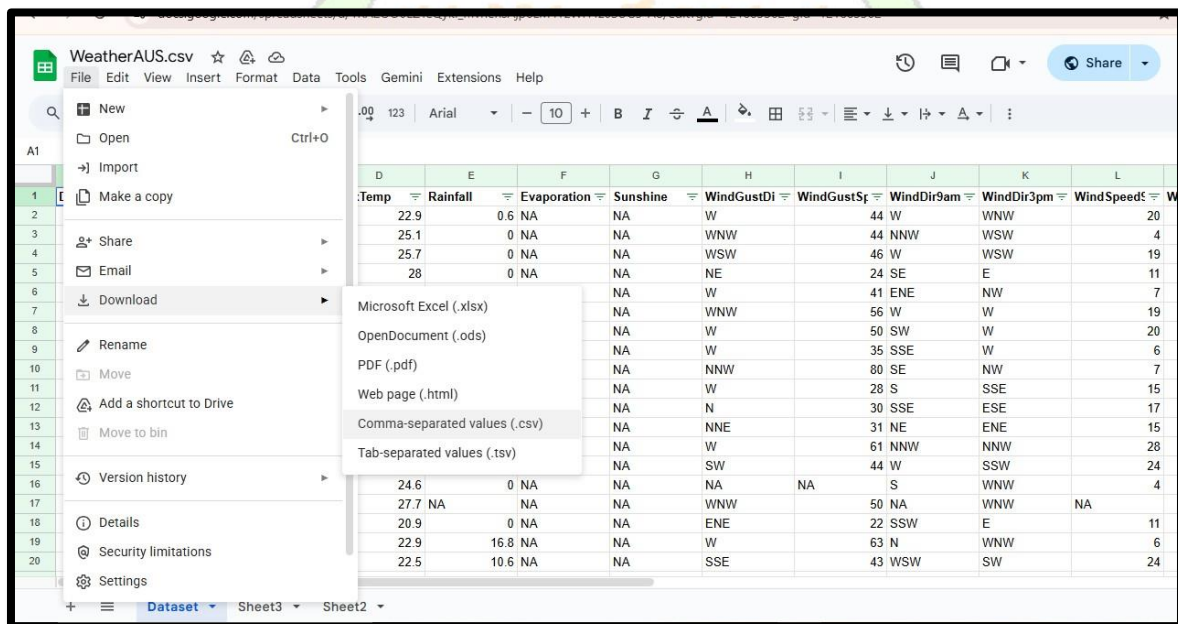
6.2 DATA COLLECTION:

Machine Learning (ML) depends heavily on data. Without data, it is impossible for an “AI” system to learn and make predictions. Data is the most crucial component that enables algorithms to identify patterns, learn relationships, and generate meaningful outputs.

DOWNLOAD THE DATASET:

Here’s how to download it:

1. Open the Google Sheets link.
2. Click File → Download → Comma-Separated Values (.csv).
3. Upload the downloaded CSV file in this chat.



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	4 MARKS

DATA PRE-PROCESSING:

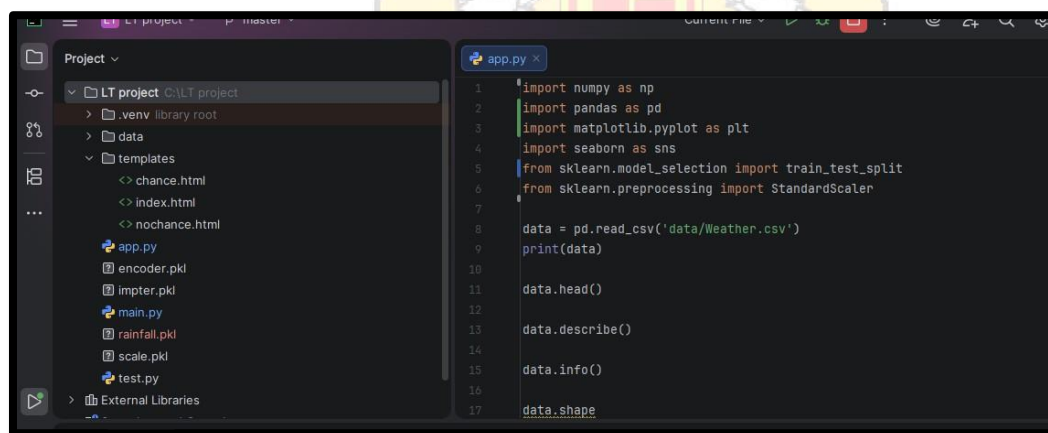
Data Pre-processing is one of the most important steps in any Machine Learning project. Raw data cannot be directly used for analysis or model building. It must be cleaned, organized, and transformed into a structured format.

Data Pre-processing in this project includes the following main tasks:

1. Importing the Libraries:

Before working with the dataset, necessary Python libraries must be imported. These libraries provide functions required for data handling, visualization, and model building.

- import numpy as np
- import pandas as pd
- import matplotlib.pyplot as plt
- import seaborn as sns
- from sklearn.model_selection import train_test_split
- from sklearn.preprocessing import StandardScaler

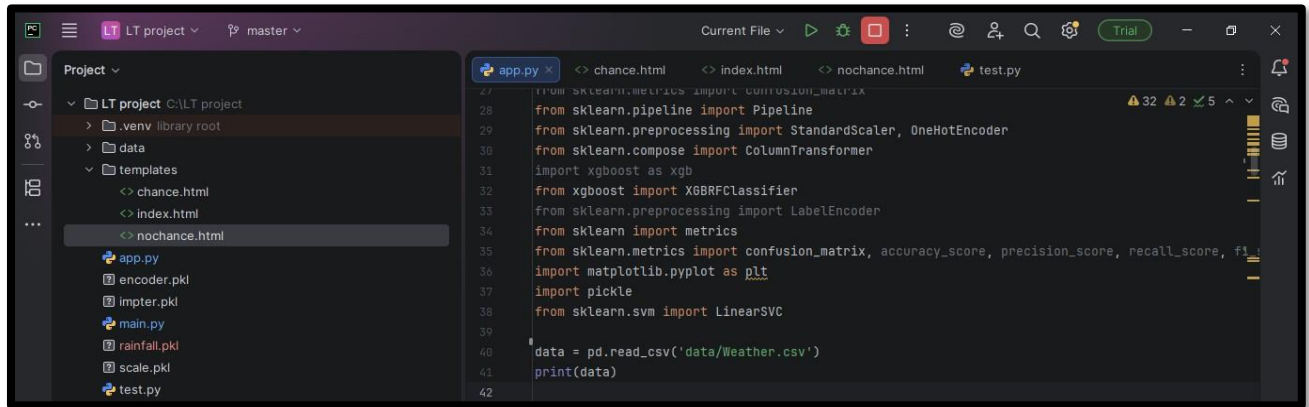


```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 data = pd.read_csv('data/Weather.csv')
9 print(data)
10
11 data.head()
12
13 data.describe()
14
15 data.info()
16
17 data.shape
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

IMPORTING THE DATASET:

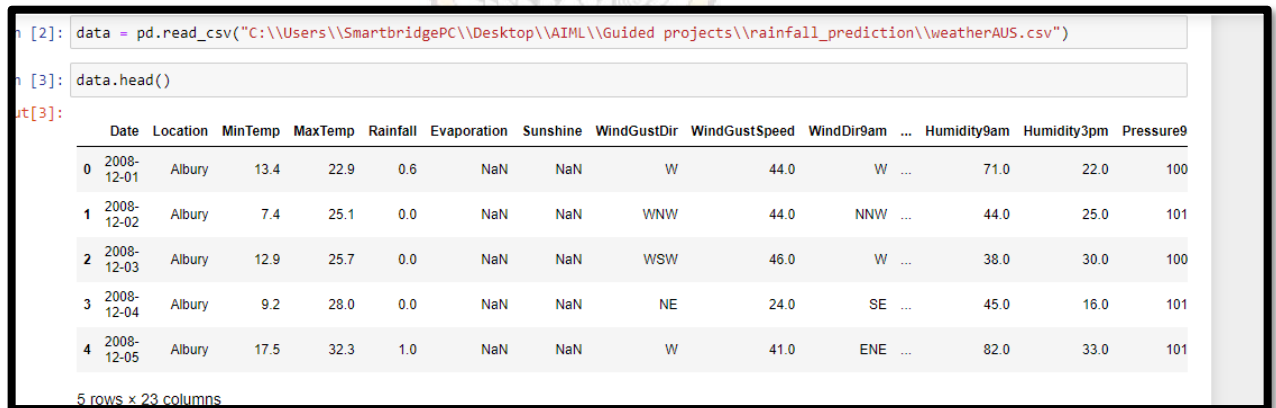
- Now we have to import the csv file on app.py
- Just like below image



```
27 from sklearn.metrics import confusion_matrix
28 from sklearn.pipeline import Pipeline
29 from sklearn.preprocessing import StandardScaler, OneHotEncoder
30 from sklearn.compose import ColumnTransformer
31 import xgboost as xgb
32 from xgboost import XGBRFClassifier
33 from sklearn.preprocessing import LabelEncoder
34 from sklearn import metrics
35 from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
36 import matplotlib.pyplot as plt
37 import pickle
38 from sklearn.svm import LinearSVC
39
40 data = pd.read_csv('data/Weather.csv')
41 print(data)
42
```

ANALYSE THE DATA:

- head () method is used to return top n (5 by default) rows of a Data Frame or



```
[2]: data = pd.read_csv("C:\\Users\\SmartbridgePC\\Desktop\\AI\\ML\\Guided projects\\rainfall_prediction\\weatherAUS.csv")
[3]: data.head()
```

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity9am	Humidity3pm	Pressure9
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	...	71.0	22.0	100
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	...	44.0	25.0	101
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	...	38.0	30.0	100
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	...	45.0	16.0	101
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	...	82.0	33.0	101

5 rows x 23 columns

series.

- Describe () method computes a summary of statistics like count, mean, standard deviation, min, max and quartile values.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

The output is as shown below

```
In [ ]: data.describe() # gives the descriptive statistics of the data
```

	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustSpeed	WindSpeed9am	WindSpeed3pm	Humidity9am	Humidity3pm
count	143975.000000	144199.000000	142199.000000	82670.000000	75625.000000	135197.000000	143693.000000	142398.000000	142806.000000	140953.000000
mean	12.194034	23.221348	2.360918	5.468232	7.611178	40.035230	14.043426	18.662657	68.880831	51.5391
std	6.398495	7.119049	8.478060	4.193704	3.785483	13.607062	8.915375	8.809800	19.029164	20.7959
min	-8.500000	-4.800000	0.000000	0.000000	0.000000	6.000000	0.000000	0.000000	0.000000	0.000000
25%	7.600000	17.900000	0.000000	2.600000	4.800000	31.000000	7.000000	13.000000	57.000000	37.000000
50%	12.000000	22.600000	0.000000	4.800000	8.400000	39.000000	13.000000	19.000000	70.000000	52.000000
75%	16.900000	28.200000	0.800000	7.400000	10.600000	48.000000	19.000000	24.000000	83.000000	66.000000
max	33.900000	48.100000	371.000000	145.000000	14.500000	135.000000	130.000000	87.000000	100.000000	100.000000

- From the data we infer that there are only decimal values and no categorical values info () gives information about the data.

```
In [ ]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                   145460 non-null object
1   Location               145460 non-null object
2   MinTemp               143975 non-null float64
3   MaxTemp               144199 non-null float64
4   Rainfall              142199 non-null float64
5   Evaporation           82670 non-null  float64
6   Sunshine              75625 non-null  float64
7   WindGustDir           135134 non-null object
8   WindGustSpeed         135197 non-null float64
9   WindDir9am            134894 non-null object
10  WindDir3pm            141232 non-null object
11  WindSpeed9am          143693 non-null float64
12  WindSpeed3pm          142398 non-null float64
13  Humidity9am           142806 non-null float64
14  Humidity3pm           140953 non-null float64
15  Pressure9am           130395 non-null float64
16  Pressure3pm           130432 non-null float64
17  Cloud9am              89572 non-null  float64
18  Cloud3pm              86102 non-null  float64
19  Temp9am               143693 non-null float64
20  Temp3pm               141851 non-null float64
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- data.shape: give the information about the dimension of the dataset, such as no. of rows and no. of columns

```
data.shape # gives the dimensions of the data  
(145460, 19)
```

Handling Missing Values:

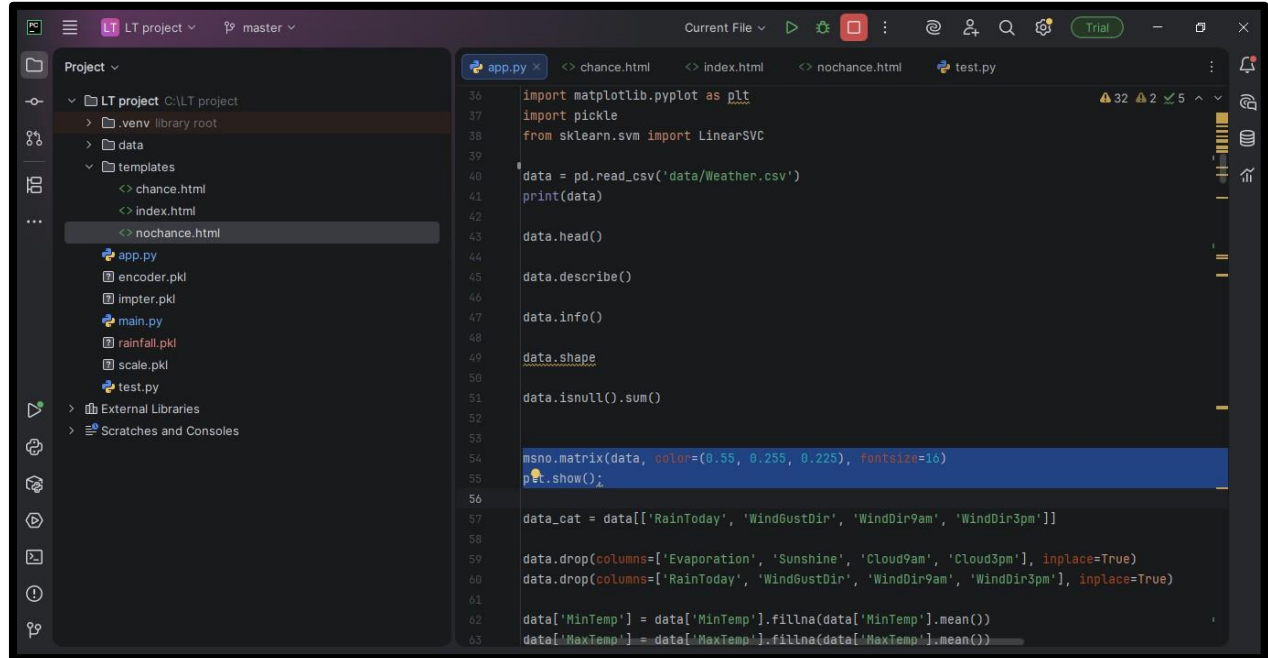
1. After loading it is important to check the complete information of data as it can indicate many of the hidden information such as null values in a column or a row
2. Check whether any null values are there or not. if it is present then the following can be done,

```
5]: data.isnull().sum()  
5]: Date           0  
   Location         0  
   MinTemp        1485  
   MaxTemp        1261  
   Rainfall       3261  
   Evaporation    62790  
   Sunshine       69835  
   WindGustDir    10326  
   WindGustSpeed  10263  
   WindDir9am     10566  
   WindDir3pm     4228  
   WindSpeed9am   1767  
   WindSpeed3pm   3062  
   Humidity9am    2654  
   Humidity3pm    4507  
   Pressure9am    15065  
   Pressure3pm    15028  
   Cloud9am       55888  
   Cloud3pm       59358  
   Temp9am        1767  
   Temp3pm        3609  
   RainToday      3261  
   RainTomorrow   3267  
   dtype: int64
```

3. **Missing matrix:** It is a way of representing the data in the 2-D form. It gives a colored visual summary of the data, missing no matrix is used to identify the missing data in the dataset, which is represented as white color gaps

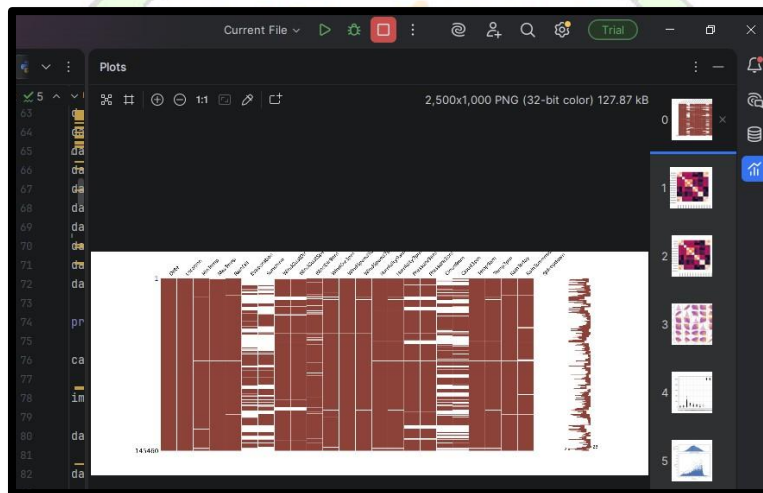
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

We have to paste the code



```
36 import matplotlib.pyplot as plt
37 import pickle
38 from sklearn.svm import LinearSVC
39
40 data = pd.read_csv('data/Weather.csv')
41 print(data)
42
43 data.head()
44
45 data.describe()
46
47 data.info()
48
49 data.shape
50
51 data.isnull().sum()
52
53
54 msno.matrix(data, color=(0.55, 0.255, 0.225), fontsize=10)
55 plt.show()
56
57 data_cat = data[['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm']]
58
59 data.drop(columns=['Evaporation', 'Sunshine', 'Cloud9am', 'Cloud3pm'], inplace=True)
60 data.drop(columns=['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm'], inplace=True)
61
62 data['MinTemp'] = data['MinTemp'].fillna(data['MinTemp'].mean())
63 data['MaxTemp'] = data['MaxTemp'].fillna(data['MaxTemp'].mean())
```

The output should be like this:



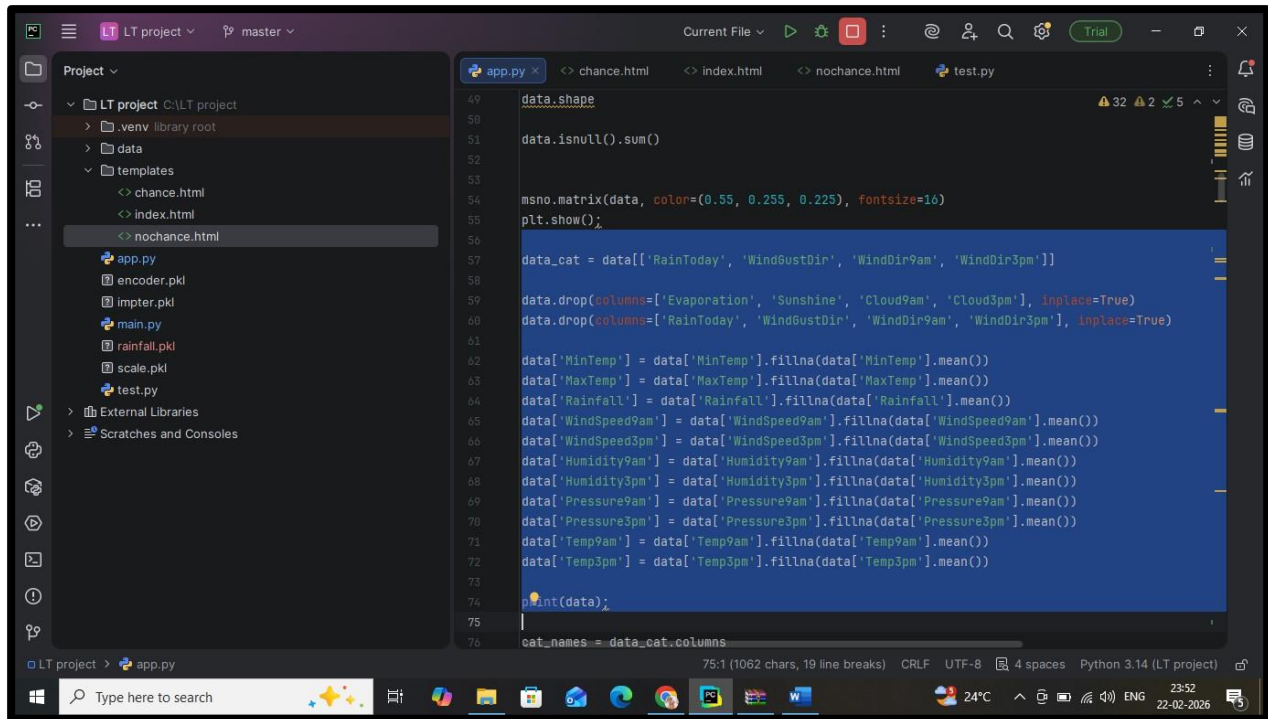
4. Imputing data using the Imputation method in sklearn.

Simple imputer

- Filling Nan values with mean, median, and mode using fillna () method.
- We are using the mean method to fill the missing data, where the data type of the columns is numeric,
- We are using the simple imputer method, which is in sklearn library from the module impute.

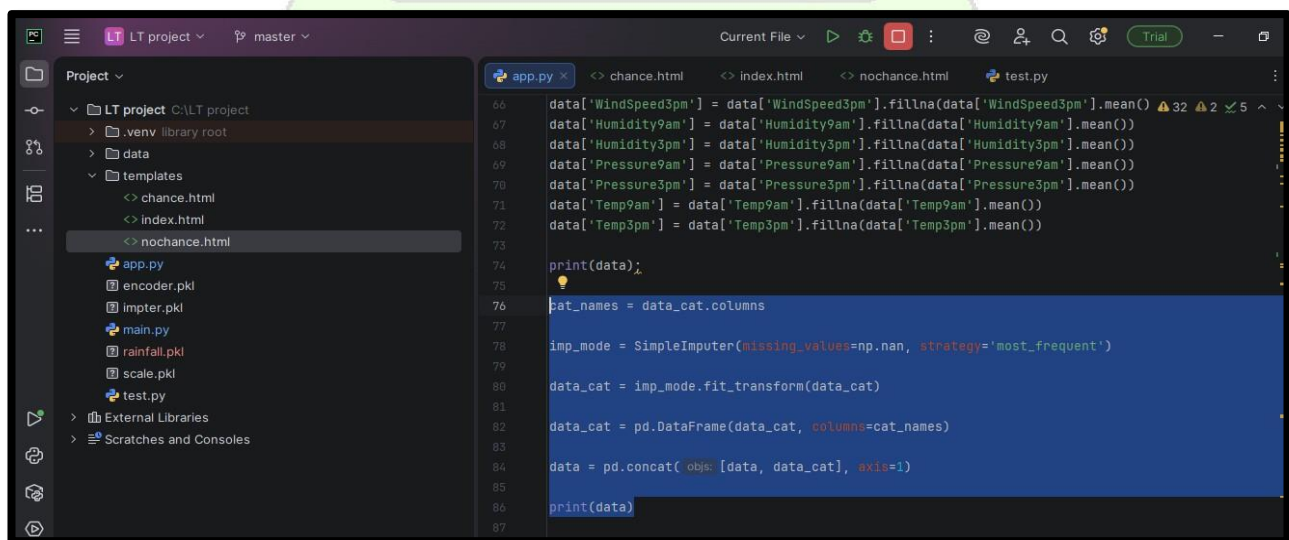
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- In impute method, for filling missing values of categorical data type we are using the most frequent method



```
49 data.shape
50 data.isnull().sum()
51
52
53
54 msno.matrix(data, color=(0.55, 0.255, 0.225), fontsize=16)
55 plt.show()
56
57 data_cat = data[['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm']]
58
59 data.drop(columns=['Evaporation', 'Sunshine', 'Cloud9am', 'Cloud3pm'], inplace=True)
60 data.drop(columns=['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm'], inplace=True)
61
62 data['MinTemp'] = data['MinTemp'].fillna(data['MinTemp'].mean())
63 data['MaxTemp'] = data['MaxTemp'].fillna(data['MaxTemp'].mean())
64 data['Rainfall'] = data['Rainfall'].fillna(data['Rainfall'].mean())
65 data['WindSpeed9am'] = data['WindSpeed9am'].fillna(data['WindSpeed9am'].mean())
66 data['WindSpeed3pm'] = data['WindSpeed3pm'].fillna(data['WindSpeed3pm'].mean())
67 data['Humidity9am'] = data['Humidity9am'].fillna(data['Humidity9am'].mean())
68 data['Humidity3pm'] = data['Humidity3pm'].fillna(data['Humidity3pm'].mean())
69 data['Pressure9am'] = data['Pressure9am'].fillna(data['Pressure9am'].mean())
70 data['Pressure3pm'] = data['Pressure3pm'].fillna(data['Pressure3pm'].mean())
71 data['Temp9am'] = data['Temp9am'].fillna(data['Temp9am'].mean())
72 data['Temp3pm'] = data['Temp3pm'].fillna(data['Temp3pm'].mean())
73
74 print(data)
75
76 cat_names = data_cat.columns
```

- From the heat map, we see that there are missing values in the dataset.



```
66 data['WindSpeed3pm'] = data['WindSpeed3pm'].fillna(data['WindSpeed3pm'].mean())
67 data['Humidity9am'] = data['Humidity9am'].fillna(data['Humidity9am'].mean())
68 data['Humidity3pm'] = data['Humidity3pm'].fillna(data['Humidity3pm'].mean())
69 data['Pressure9am'] = data['Pressure9am'].fillna(data['Pressure9am'].mean())
70 data['Pressure3pm'] = data['Pressure3pm'].fillna(data['Pressure3pm'].mean())
71 data['Temp9am'] = data['Temp9am'].fillna(data['Temp9am'].mean())
72 data['Temp3pm'] = data['Temp3pm'].fillna(data['Temp3pm'].mean())
73
74 print(data)
75
76 cat_names = data_cat.columns
77
78 imp_mode = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
79
80 data_cat = imp_mode.fit_transform(data_cat)
81
82 data_cat = pd.DataFrame(data_cat, columns=cat_names)
83
84 data = pd.concat([data, data_cat], axis=1)
85
86 print(data)
87
```

DATA VISUALIZATION:

1. Correlation

- data.Corr() gives the correlation between the columns
- Correlation is a statistical term describing the degree to which two variables move in coordination with one another.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- If the two variables move in the same direction, then those variables are said to have a positive correlation.
- If they move in opposite directions, then they have a negative correlation.
- We have to use this code for Correlation

```
data.corr()
```

	Location	MinTemp	MaxTemp	Rainfall	WindGustSpeed	WindSpeed9am	WindSpeed3pm	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm
Location	1.000000	-0.006195	-0.020490	-0.003456	0.069158	0.077025	0.064150	-0.002063	0.011040	0.036480	0.046338
MinTemp	-0.006195	1.000000	0.733919	0.103315	0.173361	0.174938	0.174187	-0.232382	0.005930	-0.424336	-0.424336
MaxTemp	-0.020490	0.733919	1.000000	-0.074204	0.066298	0.014580	0.050374	-0.499789	-0.499714	-0.309077	-0.309077
Rainfall	-0.003456	0.103315	-0.074204	1.000000	0.127263	0.085981	0.056766	0.221395	0.249617	-0.159677	-0.159677
WindGustSpeed	0.069158	0.173361	0.066298	0.127263	1.000000	0.577790	0.658334	-0.209226	-0.025751	-0.426586	-0.426586
WindSpeed9am	0.077025	0.174938	0.014580	0.085981	0.577790	1.000000	0.513067	-0.269031	-0.031001	-0.215159	-0.215159
WindSpeed3pm	0.064150	0.174187	0.050374	0.056766	0.658334	0.513067	1.000000	-0.144264	0.015782	-0.277469	-0.277469
Humidity9am	-0.002063	-0.232382	-0.499789	0.221395	-0.209226	-0.269031	-0.144264	1.000000	0.659865	0.131593	0.131593
Humidity3pm	0.011040	0.005930	-0.499714	0.249617	-0.025751	-0.031001	0.015782	0.659865	1.000000	-0.025781	-0.025781
Pressure9am	0.036480	-0.424336	-0.309077	-0.159677	-0.426586	-0.215159	-0.277469	0.131593	-0.025781	1.000000	0.959878
Pressure3pm	0.046338	-0.434021	-0.397412	-0.120366	-0.384622	-0.165038	-0.239725	0.176164	0.048554	0.959878	1.000000
Temp9am	-0.015597	0.897998	0.880085	0.011385	0.146761	0.128769	0.162143	-0.471141	-0.217573	-0.397746	-0.397746
Temp3pm	-0.022712	0.699824	0.969733	-0.077555	0.032245	0.005023	0.028444	-0.492441	-0.555778	-0.266291	-0.266291
RainToday	-0.004911	0.055644	-0.226474	0.500279	0.148264	0.100563	0.078361	0.348843	0.370561	-0.179226	-0.179226
WindGustDir	-0.005055	-0.136317	-0.212210	0.044855	0.137914	0.009762	0.084012	0.067956	0.063964	-0.120630	-0.120630
WindDir9am	-0.004434	-0.029638	-0.212589	0.085140	0.074638	0.109916	0.111072	0.088842	0.148893	-0.050314	-0.050314
WindDir3pm	0.008325	-0.158958	-0.181344	0.047905	0.136674	0.050200	0.089824	0.026077	-0.007182	-0.133633	-0.133633

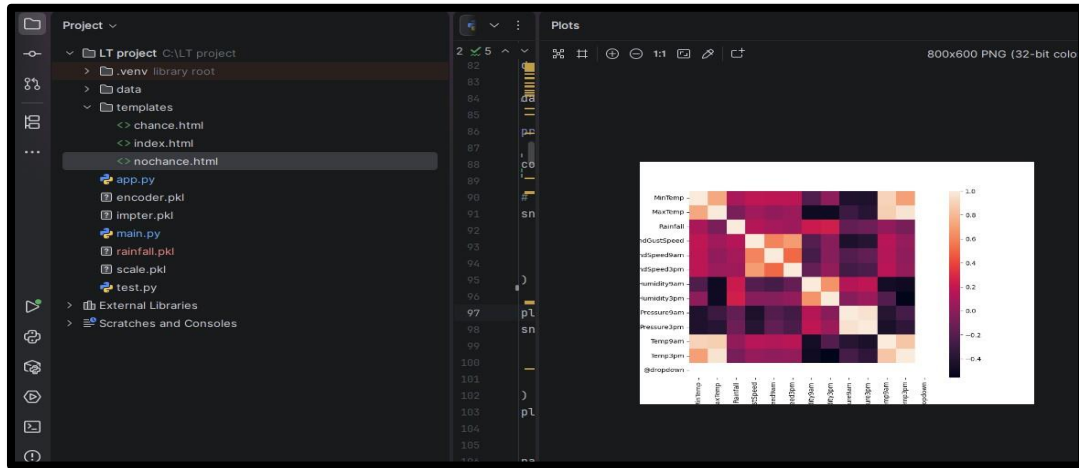
2. Heat-map:

- A heatmap is a graphical representation of data that uses a system of color-coding to represent different values. It is used to identify the correlation between the columns using a visual manner.

```
app.py x < chance.html < index.html < nochange.html < test.py
81
82 data_cat = pd.DataFrame(data_cat, columns=cat_names)
83
84 data = pd.concat([data, data_cat], axis=1)
85
86 print(data)
87
88 cor = data.corr(numeric_only=True)
89
90 # heatmap (same style as image)
91 sns.heatmap(
92     data=cor,
93     xticklabels=cor.columns.values,
94     yticklabels=cor.columns.values
95 )
96
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Output:

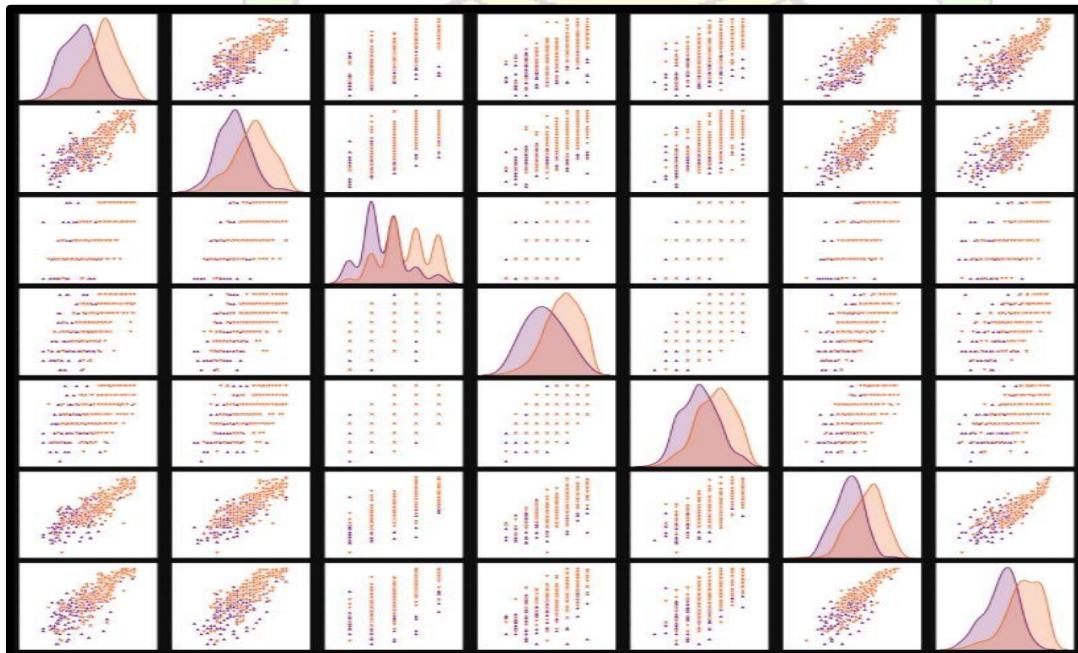


2. Pair Plot:

- Plot pairwise relationships in a dataset.

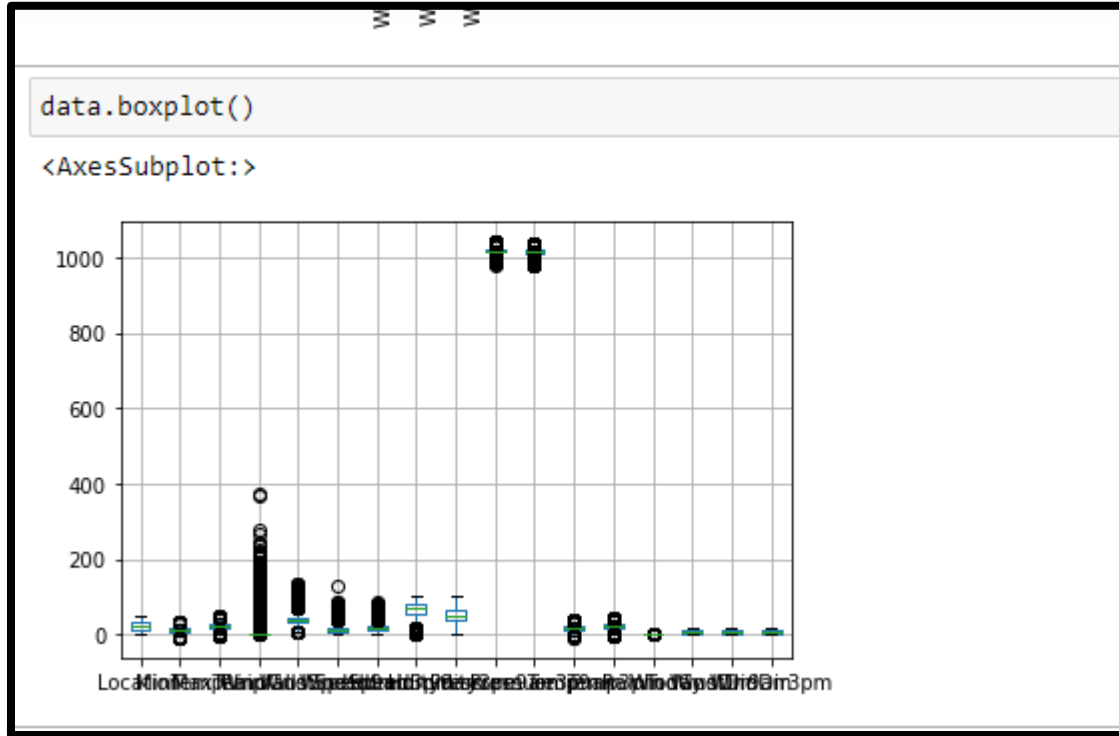
```
plt.figure(figsize=(8, 6))
sns.heatmap(
    cor,
    xticklabels=cor.columns.values,
    yticklabels=cor.columns.values
)
plt.show()
```

The output is as shown below



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

3. Box Plot:



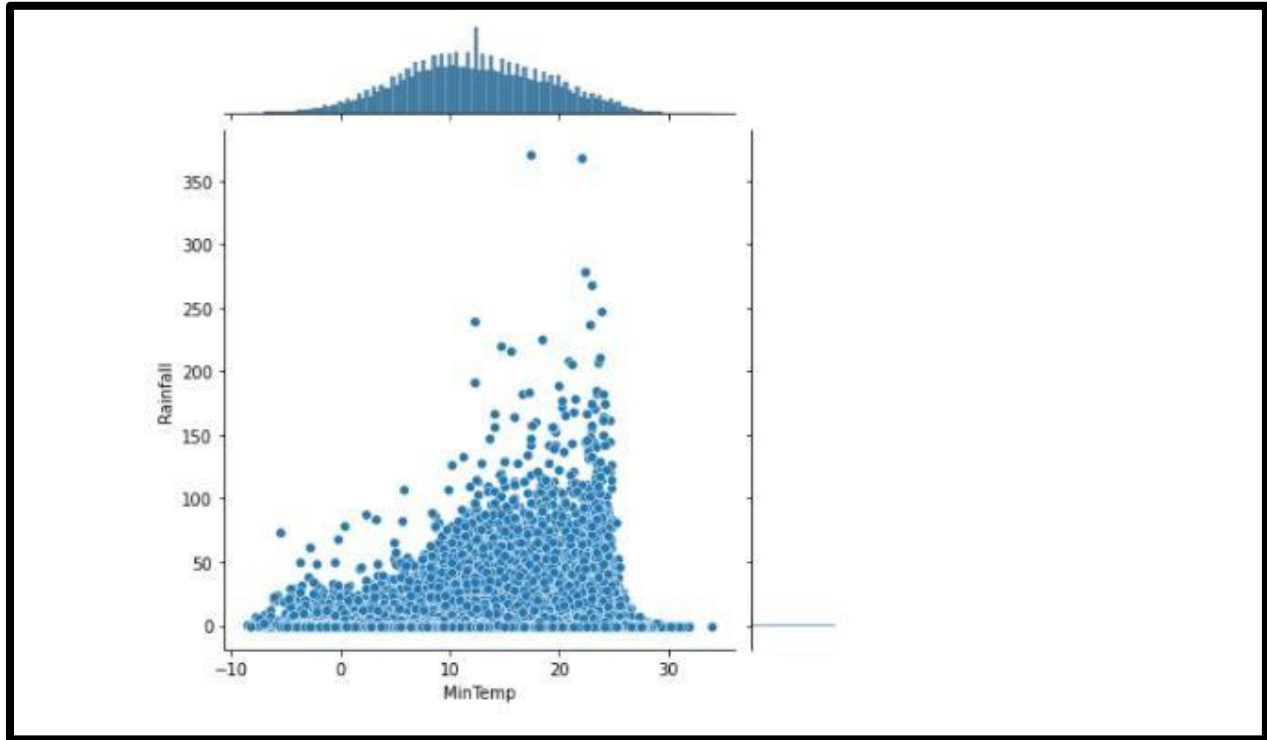
4. Joint plot:

- We have to paste code.

```
app.py |> chance.html |> index.html |> nochance.html |> test.py
123 hue='RainToday',
124 diag_kind='kde',
125 palette=['purple', 'orange'],
126 markers=['^', 'v'],
127 plot_kws={'alpha': 0.6, 's': 25},
128 diag_kws={'fill': True}
129 )
130
131 plt.show()
132
133 data.boxplot();
134 plt.show();
135
136 x = data["MinTemp"]
137 y = data["Rainfall"]
138
139 fig = plt.figure(figsize=(8, 8))
140 grid = plt.GridSpec(nrows=4, ncols=4, hspace=0.3, wspace=0.3)
141
142 ax_scatter = fig.add_subplot(grid[1:4, 0:3])
143 ax_scatter.scatter(x, y, alpha=0.6)
144 ax_scatter.set_xlabel("MinTemp")
145 ax_scatter.set_ylabel("Rainfall")
146 ax_scatter.set_title("MinTemp vs Rainfall")
147
148 ax_histx = fig.add_subplot(grid[0, 0:3], sharex=ax_scatter)
149 ax_histx.hist(x, bins=30)
150 ax_histx.tick_params(axis='x', labelbottom=False)
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Output:



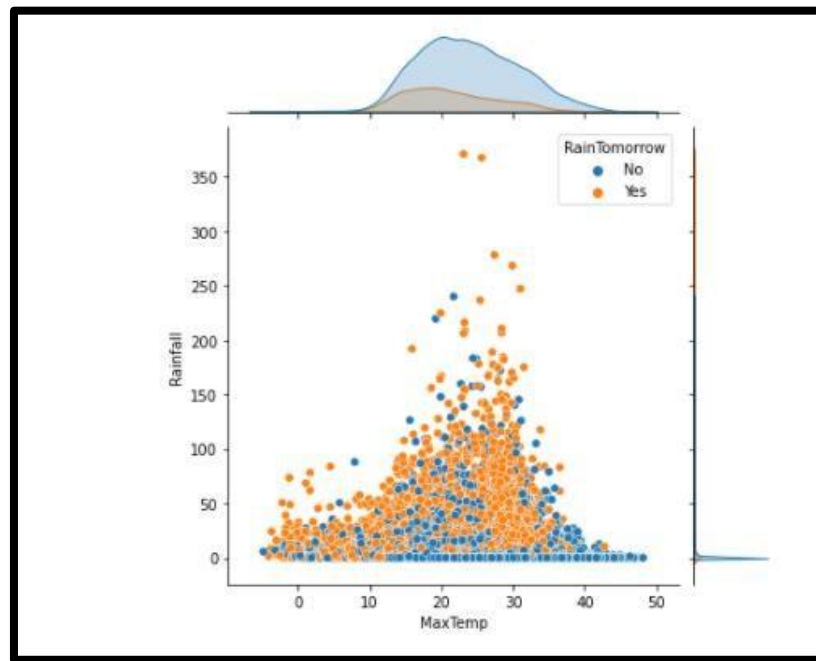
Join plot with hue function: తమసోపా జ్యోతిర్గమయ

- We have to paste code.

```
134 plt.show()
135
136 x = data["MinTemp"]
137 y = data["Rainfall"]
138
139 fig = plt.figure(figsize=(8, 8))
140 grid = plt.GridSpec(nrows=4, ncols=4, hspace=0.3, wspace=0.3)
141
142 ax_scatter = fig.add_subplot(grid[1:4, 0:3])
143 ax_scatter.scatter(x, y, alpha=0.6)
144 ax_scatter.set_xlabel("MinTemp")
145 ax_scatter.set_ylabel("Rainfall")
146 ax_scatter.set_title("MinTemp vs Rainfall")
147
148 ax_histx = fig.add_subplot(grid[0, 0:3], sharex=ax_scatter)
149 ax_histx.hist(x, bins=30)
150 ax_histx.tick_params(axis="x", labelbottom=False)
151 ax_histx.set_ylabel("Frequency")
152
153 ax_histy = fig.add_subplot(grid[1:4, 3], sharey=ax_scatter)
154 ax_histy.hist(y, bins=30, orientation="horizontal")
155 ax_histy.tick_params(axis="y", labelleft=False)
156 ax_histy.set_xlabel("Frequency")
157 plt.show()
158
159 data = data[(data["MinTemp"] > 10)]
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

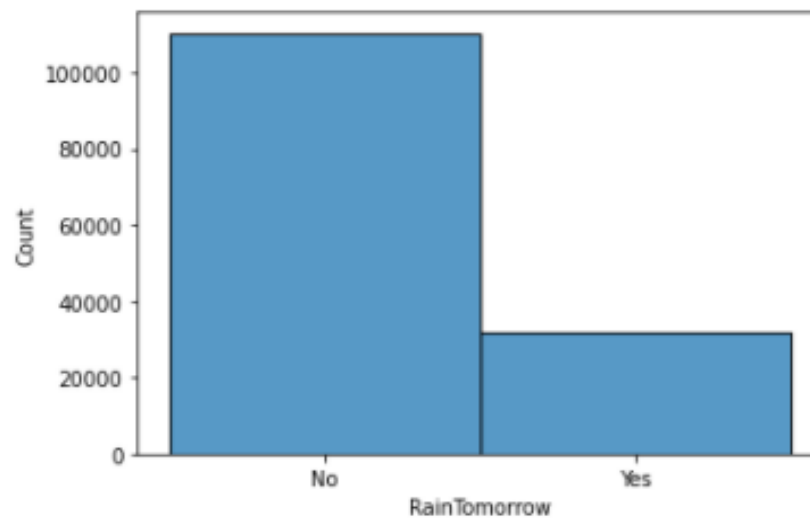
Output:



5. Hist plot:

```
In [18]: sns.histplot(data['RainTomorrow'])
```

```
Out[18]: <AxesSubplot:xlabel='RainTomorrow', ylabel='Count'>
```



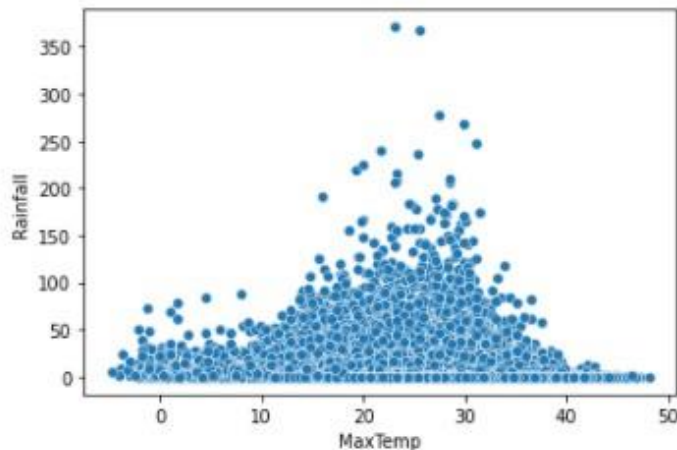
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

6. Scatter plot:

```
In [20]: sns.scatterplot(data['MaxTemp'],data['Rainfall'])
```

C:\Users\SmartbridgePC\anaconda4\lib\site-packages\seaborn_decorators.py:36: FutureWarning: yword args: x, y. From version 0.12, the only valid positional argument will be `data`. Explicit keyword will result in an error or misinterpretation.
warnings.warn(

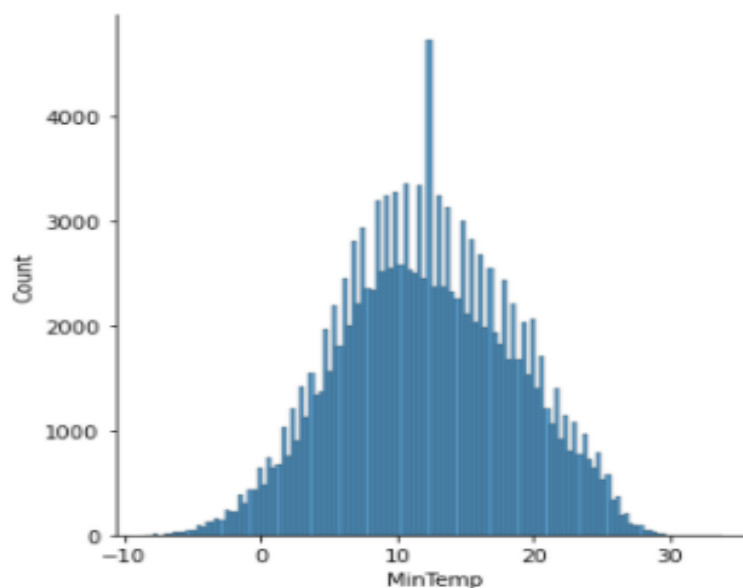
```
Out[20]: <AxesSubplot:xlabel='MaxTemp', ylabel='Rainfall'>
```



7. A Distplot or distribution plot:

```
In [19]: sns.displot(data['MinTemp'])
```

```
Out[19]: <seaborn.axisgrid.FacetGrid at 0x20b468ddfa0>
```



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

SPLITTING THE DATASET INTO DEPENDENT AND INDEPENDENT VARIABLE

- We have to paste the Below code.
- Splitting the Datasets into Dependent and Independent variable.

```
#splitting x and y values  
y = data['RainTomorrow']  
x = data.drop('RainTomorrow',axis=1)
```

Feature Scaling:

- There is huge disparity between the x values so let us use feature scaling.
- Feature scaling is a method used to normalize the range of independent variables or features of data.
- We have to paste the below code.

standardizing the data

```
: from sklearn.preprocessing import StandardScaler  
  
: #splitting x and y values  
y = data['RainTomorrow']  
x = data.drop('RainTomorrow',axis=1)  
  
: names = x.columns # Loading the names of the x_features  
  
: names  
Index(['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'WindGustSpeed',  
      'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',  
      'Pressure9am', 'Pressure3pm', 'Temp9am', 'Temp3pm', 'RainToday',  
      'WindGustDir', 'WindDir9am', 'WindDir3pm', 'year', 'month', 'day'],  
      dtype='object')  
  
: sc = StandardScaler() # initializing the standardscaler  
  
: x = sc.fit_transform(x) # fitting an transforming the data into standard  
  
: x = pd.DataFrame(x,columns=names) #scaled data turns into array format,converting them into dataframe
```

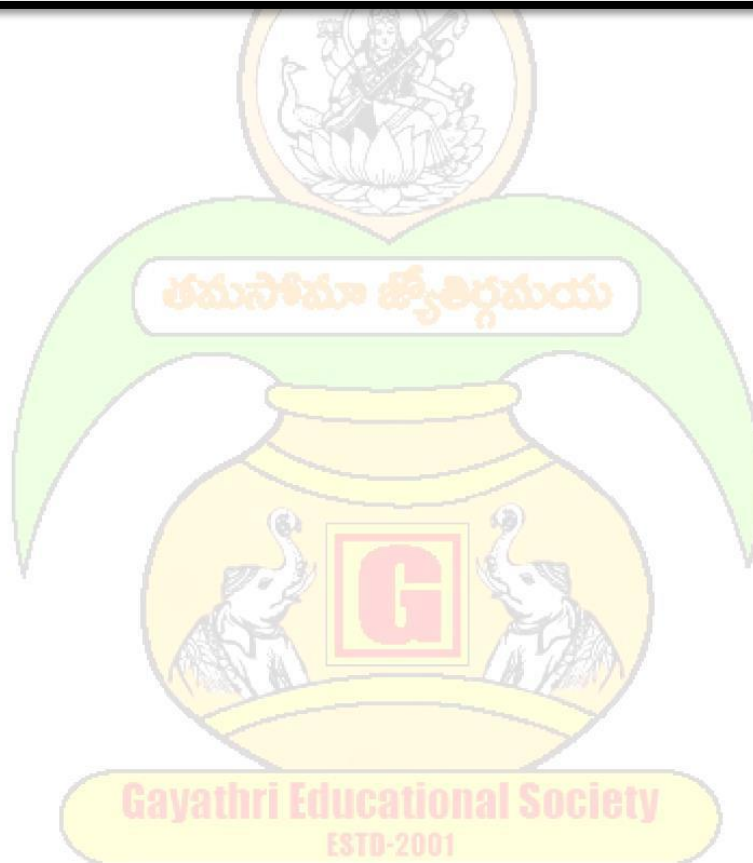
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Splitting the data into Train and Test:

- Train Dataset: Used to fit the machine learning model.
- Test Dataset: Used to evaluate the fit machine learning model.
- Now split our dataset into train set and test using train_test_split class from scikit learn library.

```
from sklearn import model_selection
```

```
x_train,x_test,y_train,y_test = model_selection.train_test_split(x,y,test_size =0.2,random_state =0)
```



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

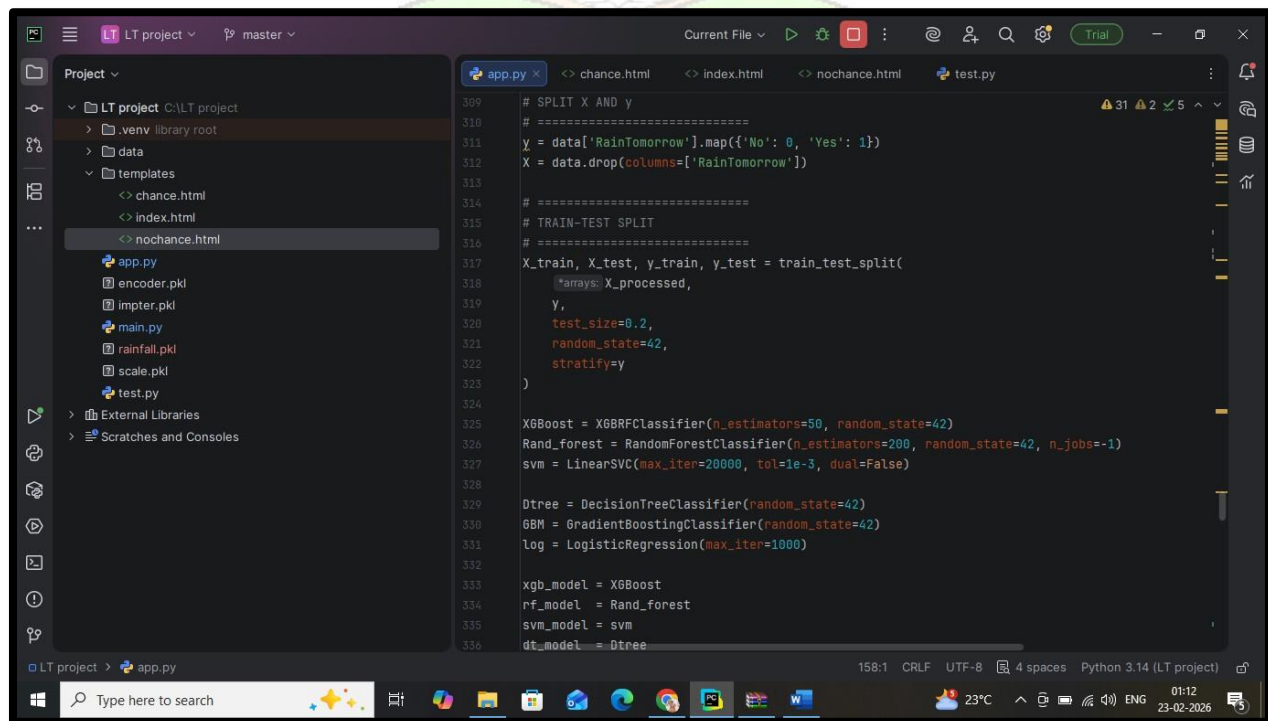
DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rainfall Data in India for Agriculture
MAXIMUM MARKS	4 MARKS

6.4 - MODEL BUILDING

Training and Testing the Model

Steps in Building the model:

- Initialize the model.
- Fit the models with x_train and y_train.
- Predict the y_train values and calculate the accuracy.
- Predict the y_test values and calculate the accuracy.



```
309 # SPLIT X AND y
310 # =====
311 y = data['RainTomorrow'].map({'No': 0, 'Yes': 1})
312 X = data.drop(columns=['RainTomorrow'])
313
314 # =====
315 # TRAIN-TEST SPLIT
316 # =====
317 X_train, X_test, y_train, y_test = train_test_split(
318     *arrays: X_processed,
319     y,
320     test_size=0.2,
321     random_state=42,
322     stratify=y
323 )
324
325 XGBoost = XGBRFClassifier(n_estimators=50, random_state=42)
326 Rand_forest = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
327 svm = LinearSVC(max_iter=20000, tol=1e-3, dual=False)
328
329 Dtree = DecisionTreeClassifier(random_state=42)
330 GBM = GradientBoostingClassifier(random_state=42)
331 log = LogisticRegression(max_iter=1000)
332
333 xgb_model = XGBoost
334 rf_model = Rand_forest
335 svm_model = svm
336 dt_model = Dtree
```

- We have to paste code.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

initializing all the models and predicting for better accuracy

```
[ ]: #Models initialization of the models
XGBoost = xgboost.XGBClassifier()
Rand_forest = sklearn.ensemble.RandomForestClassifier()
svm = sklearn.svm.SVC()
Dtree = sklearn.tree.DecisionTreeClassifier()
GBM = sklearn.ensemble.GradientBoostingClassifier()
log = sklearn.linear_model.LogisticRegression()

[ ]: # fitting the model
XGBoost.fit(x_train,y_train)
Rand_forest.fit(x_train,y_train)
svm.fit(x_train,y_train)
Dtree.fit(x_train,y_train)
GBM.fit(x_train,y_train)
log.fit(x_train,y_train)

[ ]: # predicting the train values
p1 = XGBoost.predict(x_train)
p2 = Rand_forest.predict(x_train)
p3 = svm.predict(x_train)
p4 = Dtree.predict(x_train)
p5 = GBM.predict(x_train)
p6 = log.predict(x_train)

[ ]: #checking the accuracy score
print("xgboost:",metrics.accuracy_score(y_train,p1))
print("Rand_forest:",metrics.accuracy_score(y_train,p2))
print("svm:",metrics.accuracy_score(y_train,p3))
print("Dtree:",metrics.accuracy_score(y_train,p4))
print("GBM:",metrics.accuracy_score(y_train,p5))
print("log:",metrics.accuracy_score(y_train,p6))
```

MODEL EVALUATION:

1. Accuracy score:

$$\text{Accuracy} = \frac{\text{Number of Correct predictions}}{\text{Total number of predictions made}}$$

- We have to paste the code for the value.

```
[40]: print("xgboost:",metrics.accuracy_score(y_test,t1))
      print("Rand_forest:",metrics.accuracy_score(y_test,t2))
      print("svm:",metrics.accuracy_score(y_test,t3))
      print("Dtree:",metrics.accuracy_score(y_test,t4))
      print("GBM:",metrics.accuracy_score(y_test,t5))
      print("log:",metrics.accuracy_score(y_test,t6))
```

```
xgboost: 0.8420478919793242
Rand_forest: 0.8569218326945391
svm: 0.8525967861035901
Dtree: 0.7837476704525476
GBM: 0.8499947255529379
log: 0.8418369140968388
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

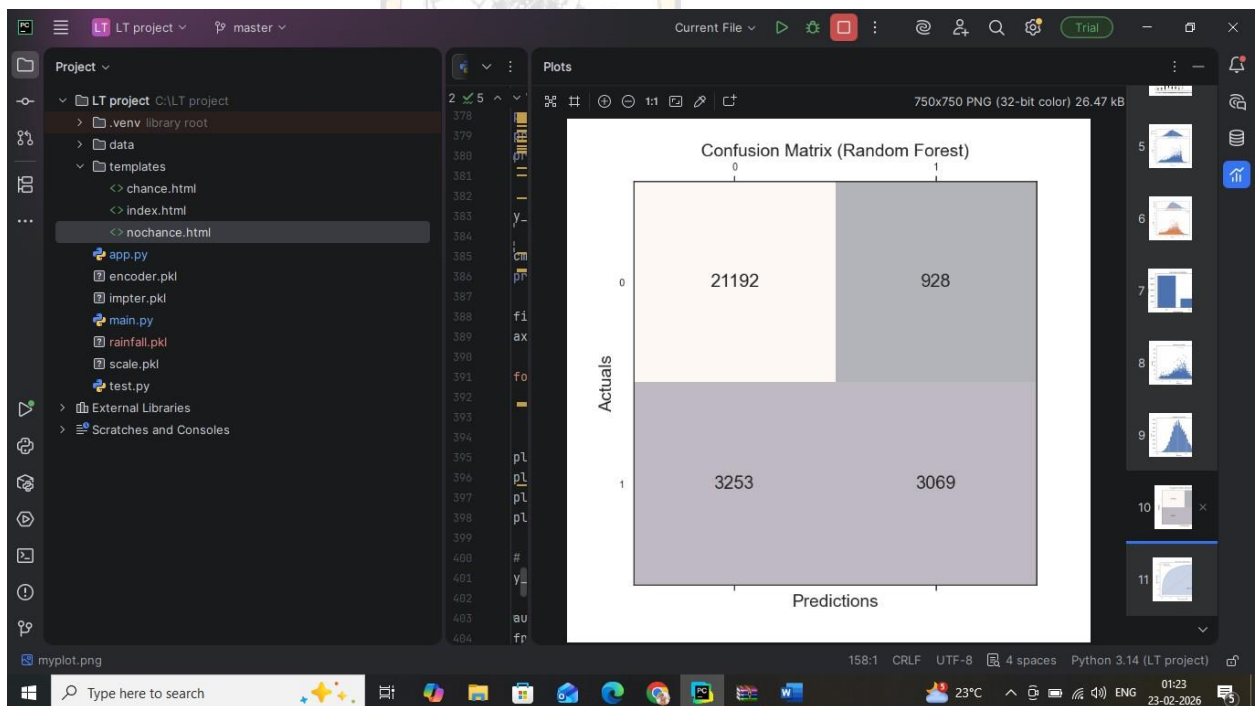
2. Confusion Matrix:

- It is a matrix representation of the results of any binary testing.

```
384 cm = confusion_matrix(y_test, y_pred)
385 print(cm)
386
387 fig, ax = plt.subplots(figsize=(7.5, 7.5))
388 ax.matshow(cm, alpha=0.3)
389
390 for i in range(cm.shape[0]):
391     for j in range(cm.shape[1]):
392         ax.text(j, i, cm[i, j], va='center', ha='center', size='xx-large')
393
394 plt.xlabel(xlabel: "Predictions", fontsize=18)
395 plt.ylabel(ylabel: "Actuals", fontsize=18)
396 plt.title(label: "Confusion Matrix (Random Forest)", fontsize=18)
397 plt.show()
398
399
```

- We have to paste the above code

Output:



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

3. Roc Auc Curve:

- We have to paste the below code for getting the roc auc curve.

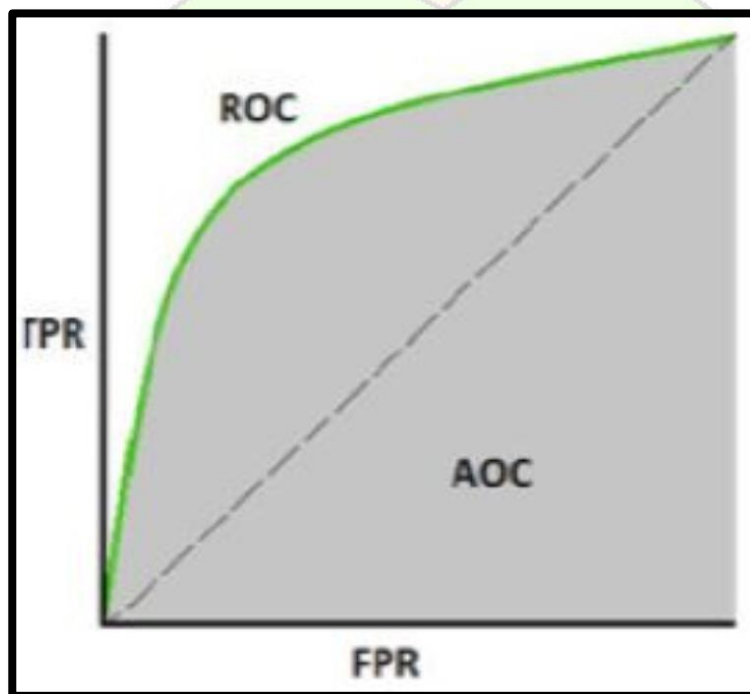
```
] : print(conf_matrix)
print("Accuracy:", Accuracy)
print("Precesion:", Precesion)
print("Recall:", Recall)
print("F1-score:", F1_score)

] : auc = metrics.roc_auc_score(y_test, y_pred)

fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)

plt.figure(figsize=(12, 10), dpi=80)
plt.axis('scaled')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.title("AUC & ROC Curve")
plt.plot(fpr, tpr, 'v')
plt.fill_between(fpr, tpr, facecolor='blue', alpha=0.8)
plt.text(1, 0.05, 'AUC = %0.4f' % auc, ha='right', fontsize=10, weight='bold', color='black')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```

Output:



EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

Save the Model:

- After building the model we have to save the model.
- We have to paste the below code for pkl files.

```
# 1. Save the actual model you trained
# In your script, this was named 'Rand_forest'
pickle.dump(Rand_forest, open('rainfall.pkl', 'wb'))

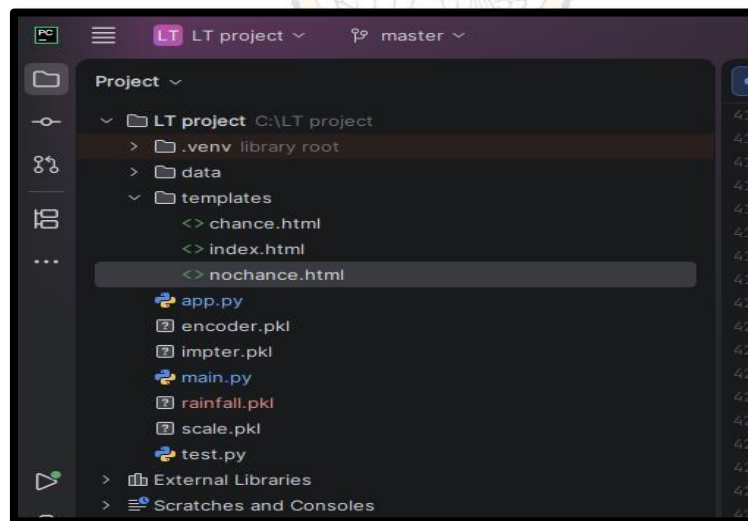
# 2. Save the preprocessor
# This replaces 'le', 'imp_mode', and 'sc' because your ColumnTransformer
# already contains the Imputer, Scaler, and OneHotEncoder combined.
pickle.dump(preprocessor, open('encoder.pkl', 'wb'))

# 3. Save the categorical imputer (Optional)
# If you specifically need the categorical imputer you named 'imp_mode'
pickle.dump(imp_mode, open('impter.pkl', 'wb'))

# This object contains your scaling logic
pickle.dump(preprocessor, open('scale.pkl', 'wb'))
```

Output:

- After running the code the pkl files created like below.



Gayathri Educational Society
ESTD-2001

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rainfall Data in India for Agriculture
MAXIMUM MARKS	4 MARKS

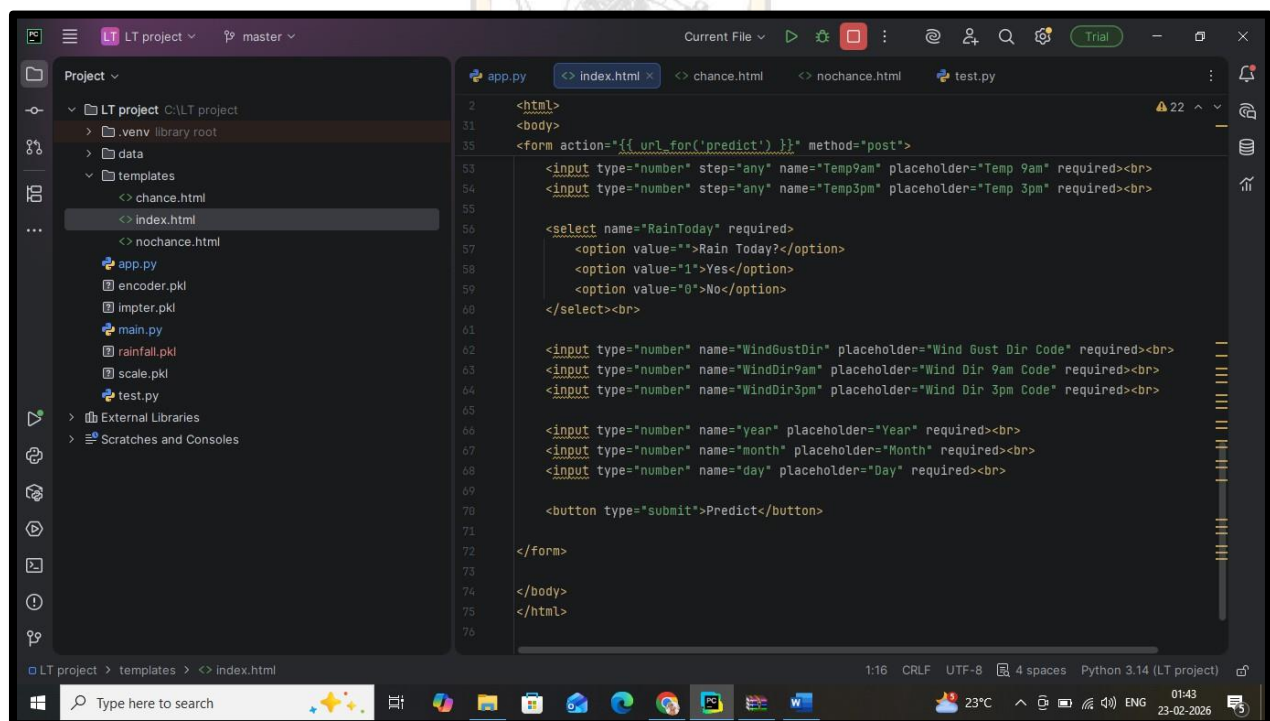
6.5 - APPLICATION BUILDING:

In our project we have 3 HTML files

1. index.html
2. chance.html
3. nochance.html

1.index.html:

- We have to paste code.
- On the html file only

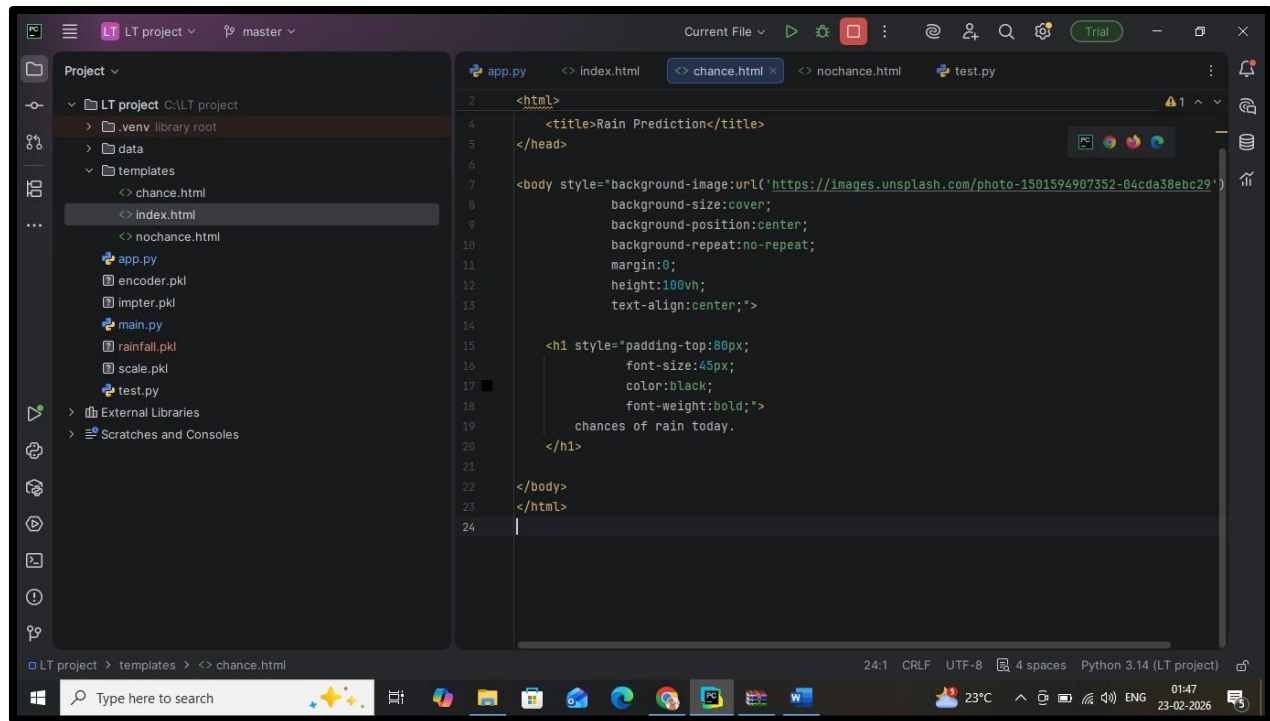


```
2 <html>
31 <body>
35 <form action="{url_for('predict')}" method="post">
53 <input type="number" step="any" name="Temp9am" placeholder="Temp 9am" required><br>
54 <input type="number" step="any" name="Temp3pm" placeholder="Temp 3pm" required><br>
55
56 <select name="RainToday" required>
57 <option value="">Rain Today?</option>
58 <option value="1">Yes</option>
59 <option value="0">No</option>
60 </select><br>
61
62 <input type="number" name="WindGustDir" placeholder="Wind Gust Dir Code" required><br>
63 <input type="number" name="WindDir9am" placeholder="Wind Dir 9am Code" required><br>
64 <input type="number" name="WindDir3pm" placeholder="Wind Dir 3pm Code" required><br>
65
66 <input type="number" name="year" placeholder="Year" required><br>
67 <input type="number" name="month" placeholder="Month" required><br>
68 <input type="number" name="day" placeholder="Day" required><br>
69
70 <button type="submit">Predict</button>
71
72 </form>
73
74 </body>
75 </html>
76
```

2. chance.html:

- Now we have to create another html file that is named as chance.html.
- In that file we have to paste the below code.

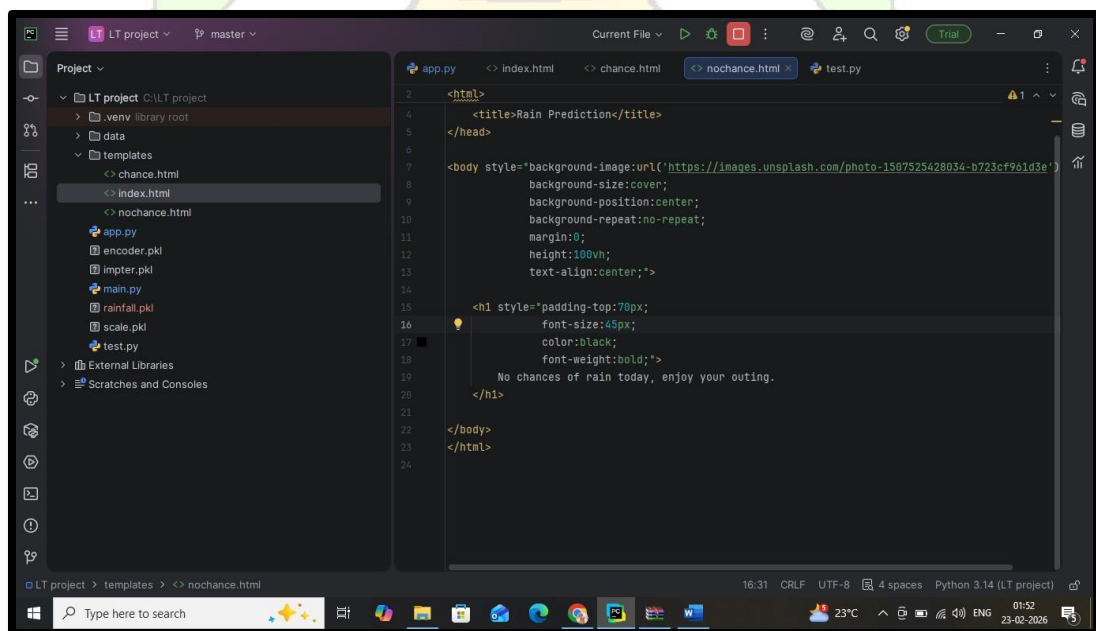
EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE



```
2 <html>
3 <title>Rain Prediction</title>
4 </head>
5
6 <body style="background-image:url('https://images.unsplash.com/photo-1501594907352-04cda38ebc29');
7   background-size:cover;
8   background-position:center;
9   background-repeat:no-repeat;
10  margin:0;
11  height:100vh;
12  text-align:center;">
13
14   <h1 style="padding-top:80px;
15     font-size:45px;
16     color:black;
17     font-weight:bold;">
18     chances of rain today.
19   </h1>
20
21 </body>
22 </html>
23
24
```

3. Nochance.html:

- Now we have to create another html file that is named as nochance.html.
- In that file we have to paste the below code.

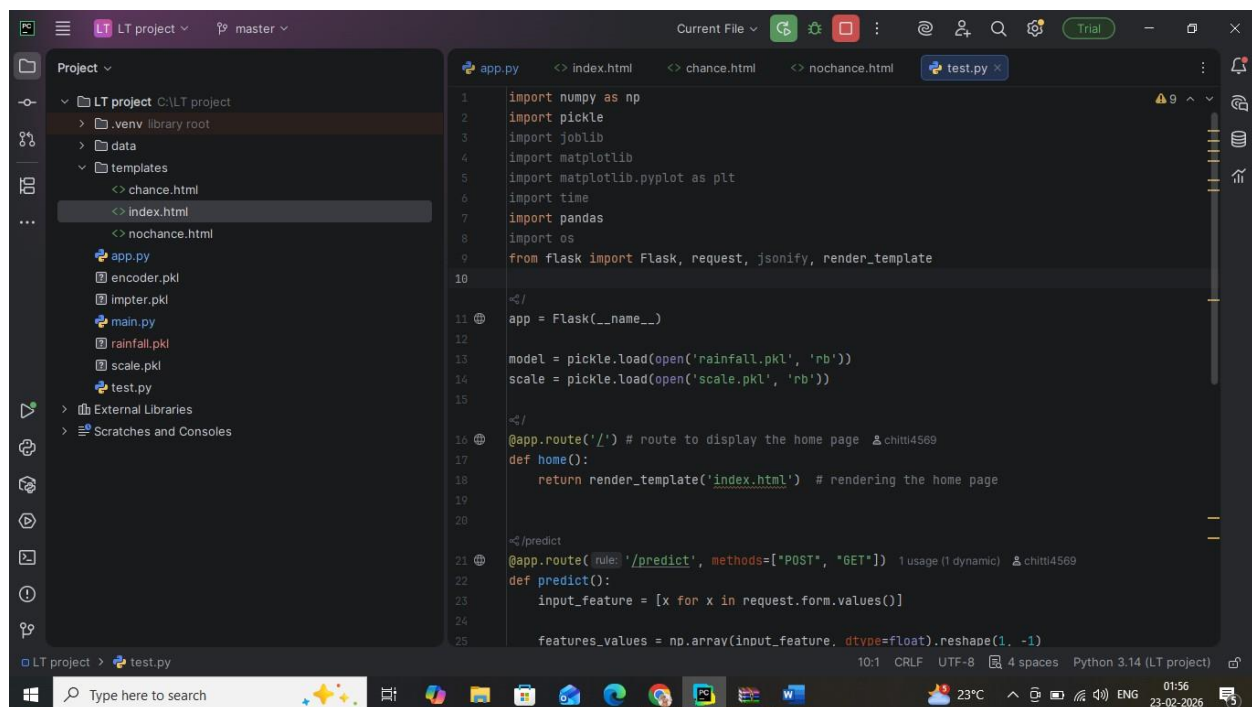


```
2 <html>
3 <title>Rain Prediction</title>
4 </head>
5
6 <body style="background-image:url('https://images.unsplash.com/photo-1507525428034-b723cf961d3e');
7   background-size:cover;
8   background-position:center;
9   background-repeat:no-repeat;
10  margin:0;
11  height:100vh;
12  text-align:center;">
13
14   <h1 style="padding-top:70px;
15     font-size:45px;
16     color:black;
17     font-weight:bold;">
18     No chances of rain today, enjoy your outing.
19   </h1>
20
21 </body>
22 </html>
23
24
```

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

MAIN PYTHON SCRIPT:

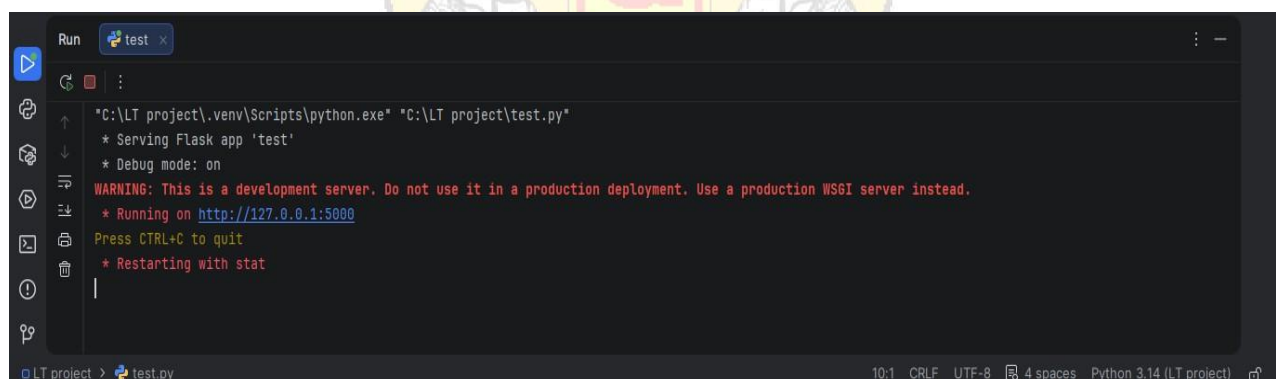
- Let us build an test.py flask file which is a web framework written in python for server-side scripting.
- Let's see step by step procedure for building the backend application.
- Now we have to create a python file named as test.py
- Paste the below code on test.py.



```
1 import numpy as np
2 import pickle
3 import joblib
4 import matplotlib
5 import matplotlib.pyplot as plt
6 import time
7 import pandas
8 import os
9 from flask import Flask, request, jsonify, render_template
10
11 app = Flask(__name__)
12
13 model = pickle.load(open('rainfall.pkl', 'rb'))
14 scale = pickle.load(open('scale.pkl', 'rb'))
15
16 @app.route('/') # route to display the home page
17 def home():
18     return render_template('index.html') # rendering the home page
19
20
21 @app.route('/predict', methods=['POST', 'GET'])
22 def predict():
23     input_feature = [x for x in request.form.values()]
24
25     features_values = np.array(input_feature, dtype=float).reshape(1, -1)
```

RUN THE APP:

- Now we have to run the code on test.py.
- Then click on the link on the run bar.



```
Run test
"C:\LT project\.venv\Scripts\python.exe" "C:\LT project\test.py"
* Serving Flask app 'test'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
```

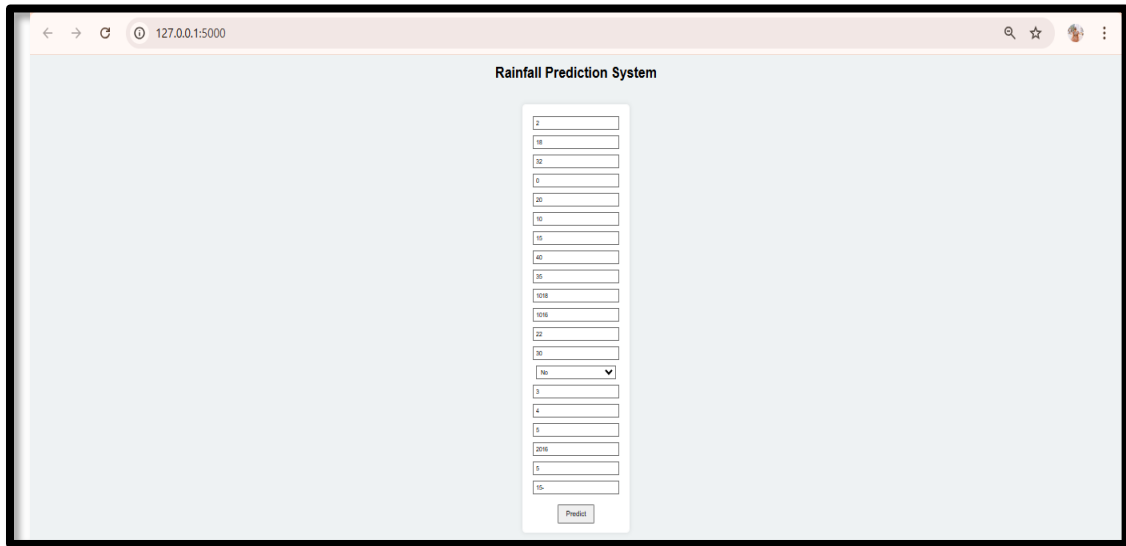
- Navigate to the localhost where you can view your web page, then it will run on local host: 5000

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

OUTPUT:

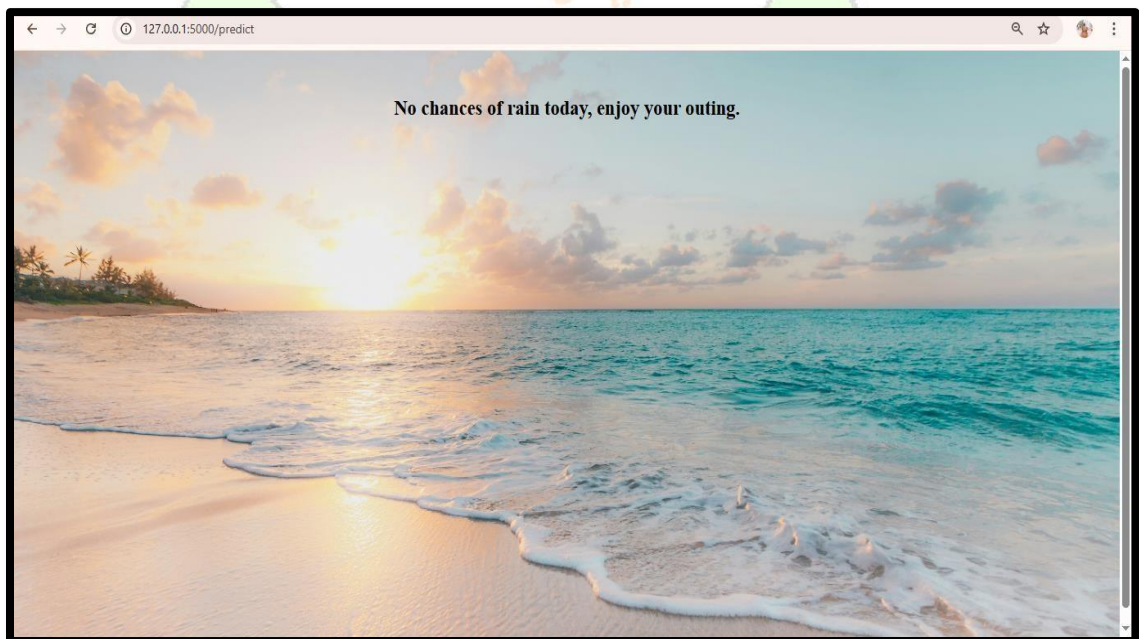
No chance of rain:

- After clicking the link, we have to fill the rain predictions system form.
- For getting the no chance of rain.



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". The page title is "Rainfall Prediction System". The form contains several input fields for numerical values: 2, 18, 12, 0, 20, 10, 15, 40, 25, 1018, 1016, 22, 10, 180 (with a dropdown arrow), 3, 4, 5, 2016, 5, and 15. A "Predict" button is located at the bottom of the form.

- Now click on predict.
- Now it opens the output.



Chances of rain:

- After clicking the link, we have to fill the rain predictions system form.

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

- For getting the chance of rain.

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The page title is 'Rainfall Prediction System'. The main content area features a vertical stack of input fields for numerical data, including values like 2, 20, 25, 10, 45, 30, 35, 65, 90, 1000, 1002, 22, 24, and 200. There is a dropdown menu with 'Yes' selected, followed by input fields for 6, 7, 5, 2010, 7, and 20. A 'Predict' button is located at the bottom of the form.

- Now click on predict.
- Now it opens the output.



Exploratory Analysis of Rainfall Data in India for Agriculture

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rainfall Data in India for Agriculture
MAXIMUM MARKS	5MARKS

CHAPTER – 7

7.1 Functional and Performance Testing

Rainfall Prediction System

6
23.5
28.2
12.6
46
18
24
92
88
1004.1
1001.6
24.7
26.1
Yes
8
7
7
2025
8
14

Predict



Exploratory Analysis of Rainfall Data in India for Agriculture

127.0.0.1:5000

Rainfall Prediction System

2

20

25

10

40

30

35

40

50

1005

1002

22

24

1000

6

7

5

2015

7

20

Predict

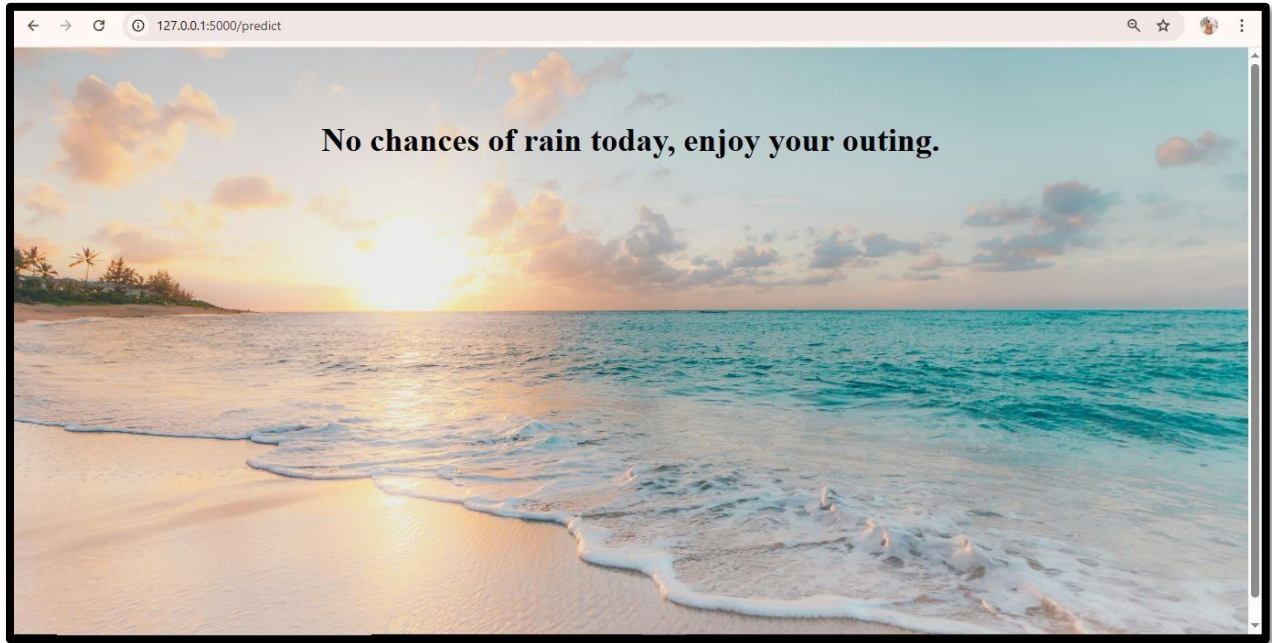
chances of rain today.

Gayathri Educational Society
ESTD-2001

CHAPTER-8

8.1 - Output Screens

1. No chance of rain:



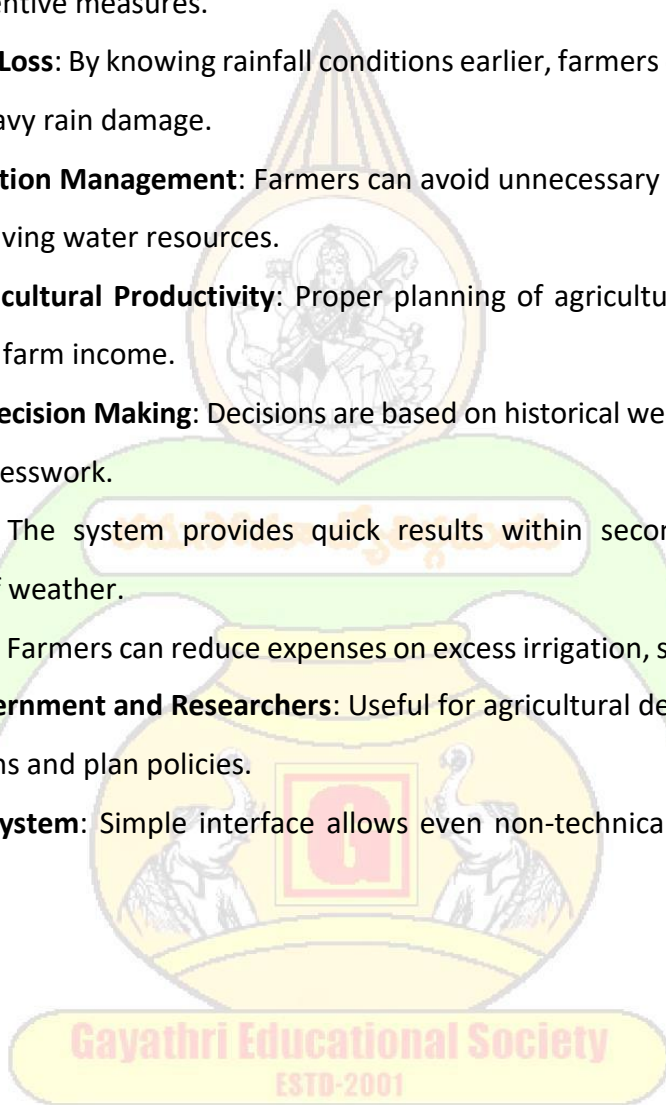
2. Chance of rain:



CHAPTER – 09

9.1 ADVANTAGES:

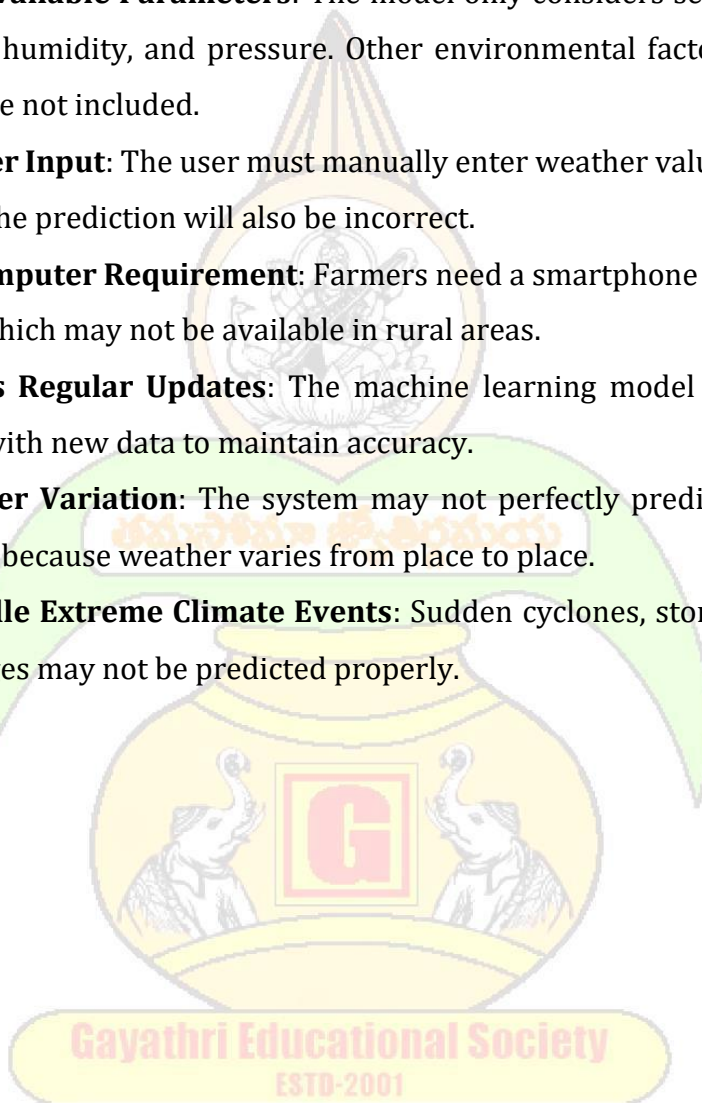
1. **Helps Farmers in Crop Planning:** Farmers can decide the correct time for sowing and harvesting based on rainfall prediction.
2. **Early Rainfall Prediction:** The system predicts whether rain will occur or not in advance, allowing preventive measures.
3. **Reduces Crop Loss:** By knowing rainfall conditions earlier, farmers can protect crops from drought or heavy rain damage.
4. **Efficient Irrigation Management:** Farmers can avoid unnecessary watering when rainfall is expected, saving water resources.
5. **Improves Agricultural Productivity:** Proper planning of agricultural activities increases crop yield and farm income.
6. **Data-Driven Decision Making:** Decisions are based on historical weather data and analysis rather than guesswork.
7. **Time Saving:** The system provides quick results within seconds, reducing manual observation of weather.
8. **Cost Effective:** Farmers can reduce expenses on excess irrigation, seeds, and fertilizers.
9. **Supports Government and Researchers:** Useful for agricultural departments to monitor rainfall patterns and plan policies.
10. **Easy to Use System:** Simple interface allows even non-technical users to operate the application.



Exploratory Analysis of Rainfall Data in India for Agriculture

9.2 - DISADVANTAGES:

1. **Depends on Data Quality:** The prediction accuracy depends on the correctness of historical weather data. Incorrect or incomplete data can give wrong results.
2. **Not 100% Accurate:** Weather is a natural phenomenon and can change suddenly. Therefore, the system cannot guarantee perfectly accurate rainfall prediction.
3. **Limited to Available Parameters:** The model only considers selected features like temperature, humidity, and pressure. Other environmental factors may also affect rainfall but are not included.
4. **Requires User Input:** The user must manually enter weather values. If wrong values are entered, the prediction will also be incorrect.
5. **Internet/Computer Requirement:** Farmers need a smartphone or computer to use the system, which may not be available in rural areas.
6. **Model Needs Regular Updates:** The machine learning model must be retrained periodically with new data to maintain accuracy.
7. **Local Weather Variation:** The system may not perfectly predict rainfall for very small regions because weather varies from place to place.
8. **Cannot Handle Extreme Climate Events:** Sudden cyclones, storms, or unexpected climate changes may not be predicted properly.



CHAPTER – 10

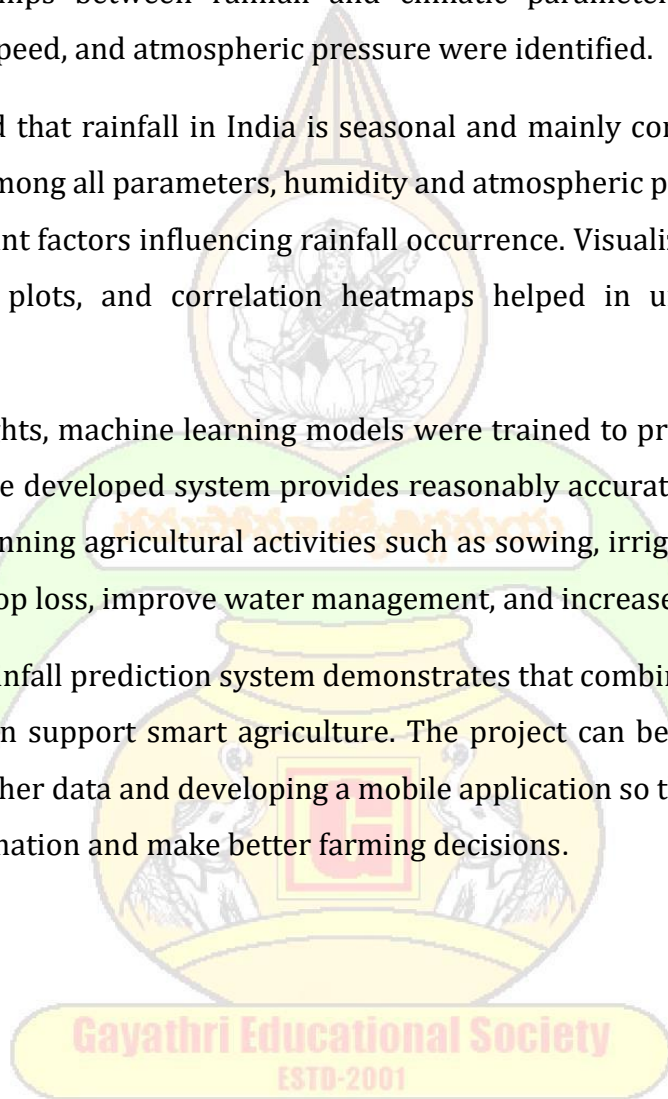
10.1 Conclusion:

The project “Exploratory Analysis of Rainfall Data in India for Agriculture” focuses on studying rainfall patterns using historical weather data and applying machine learning techniques for rainfall prediction. Through data preprocessing and exploratory data analysis, important relationships between rainfall and climatic parameters such as humidity, temperature, wind speed, and atmospheric pressure were identified.

The analysis showed that rainfall in India is seasonal and mainly concentrated during the monsoon months. Among all parameters, humidity and atmospheric pressure were found to be the most significant factors influencing rainfall occurrence. Visualization techniques like histograms, scatter plots, and correlation heatmaps helped in understanding rainfall behaviour clearly.

Based on these insights, machine learning models were trained to predict whether rainfall will occur or not. The developed system provides reasonably accurate predictions and can assist farmers in planning agricultural activities such as sowing, irrigation, and harvesting. This helps reduce crop loss, improve water management, and increase productivity.

In conclusion, the rainfall prediction system demonstrates that combining data analysis and machine learning can support smart agriculture. The project can be further improved by using real-time weather data and developing a mobile application so that farmers can easily access rainfall information and make better farming decisions.



CHAPTER – 11

11.1-FUTURE SCOPE:

The current rainfall prediction system provides useful analysis and prediction based on historical weather data. However, the system can be further improved and expanded in the future to make it more accurate and beneficial for farmers and agricultural planning.

1. **Real-Time Weather Data Integration:** The system can be connected to live weather APIs to obtain real-time temperature, humidity, and pressure values for more accurate predictions.
2. **Mobile Application Development:** A mobile app can be developed so farmers can easily access rainfall predictions directly from their smartphones.
3. **Location-Based Prediction:** The system can be enhanced to provide district-wise or village-wise rainfall prediction using GPS location.
4. **Crop Recommendation System:** Based on rainfall prediction, the system can suggest suitable crops (rice, wheat, millets, cotton, etc.) for better yield.
5. **Irrigation Advisory:** The application can recommend when to irrigate fields and how much water is required, helping in water conservation.
6. **Drought and Flood Warning System:** Advanced models can be used to detect extreme weather conditions and provide early alerts to farmers.
7. **Improved Machine Learning Models:** Deep learning and advanced AI techniques can be implemented to increase prediction accuracy.
8. **Multilingual Support:** The system can be made available in regional languages so that rural farmers can easily understand and use it.
9. **Government and Agricultural Integration:** The project can be integrated with agricultural departments to support large-scale farming decisions and policy planning.

CHAPTER-12

12.1 - Source Code

1.index.html

Code:

```

<!DOCTYPE html>
<html>
<head>
  <title>Rainfall Prediction</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #eef2f3;
      text-align: center;
    }
    form {
      background: #fff;
      padding: 20px;
      display: inline-block;
      margin-top: 30px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }
    input, select {
      margin: 6px;
      padding: 6px;
      width: 200px;
    }
    button {
      margin-top: 15px;
      padding: 10px 20px;
      font-size: 16px;
    }
  </style>
</head>
<body>

<h1>Rainfall Prediction System</h1>

<form action="{{ url_for('predict') }}" method="post">

  <input type="number" name="Location" placeholder="Location Code"
required><br>

```

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

```
<input type="number" step="any" name="MinTemp" placeholder="Min  
Temp" required><br>  
<input type="number" step="any" name="MaxTemp" placeholder="Max  
Temp" required><br>  
<input type="number" step="any" name="Rainfall" placeholder="Rainfall" required><br>
```

```
<input type="number" step="any" name="WindGustSpeed" placeholder="Wind Gust Speed" required><br>  
<input type="number" step="any" name="WindSpeed9am" placeholder="Wind Speed 9am" required><br>  
<input type="number" step="any" name="WindSpeed3pm" placeholder="Wind Speed 3pm" required><br>
```

```
<input type="number" step="any" name="Humidity9am" placeholder="Humidity 9am" required><br>  
<input type="number" step="any" name="Humidity3pm" placeholder="Humidity 3pm" required><br>
```

```
<input type="number" step="any" name="Pressure9am" placeholder="Pressure 9am" required><br>  
<input type="number" step="any" name="Pressure3pm" placeholder="Pressure 3pm" required><br>
```

```
<input type="number" step="any" name="Temp9am" placeholder="Temp  
9am" required><br>  
<input type="number" step="any" name="Temp3pm" placeholder="Temp  
3pm" required><br>
```

```
<select name="RainToday" required>  
  <option value="">Rain Today?</option>  
  <option value="1">Yes</option>  
  <option value="0">No</option>  
</select><br>
```

```
<input type="number" name="WindGustDir" placeholder="Wind Gust  
Dir Code" required><br>  
<input type="number" name="WindDir9am" placeholder="Wind Dir 9am  
Code" required><br>  
<input type="number" name="WindDir3pm" placeholder="Wind Dir 3pm  
Code" required><br>
```

```
<input type="number" name="year" placeholder="Year" required><br>  
<input type="number" name="month" placeholder="Month"  
required><br>
```

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

```
<input type="number" name="day" placeholder="Day" required><br>
```

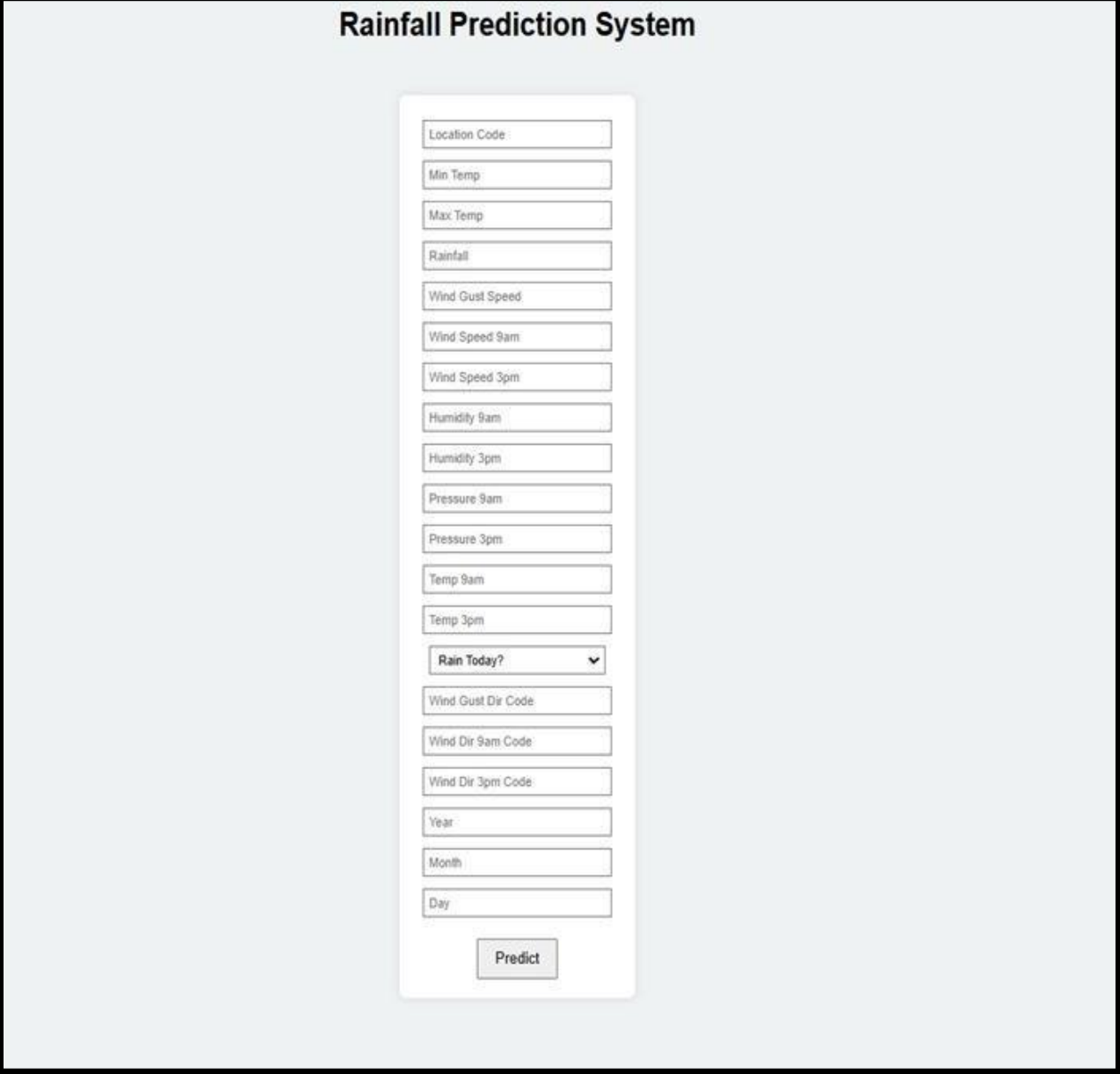
```
<button type="submit">Predict</button>
```

```
</form>
```

```
</body>
```

```
</html>
```

Output:

A screenshot of a web application titled "Rainfall Prediction System". The form is centered on a light blue background. It contains a vertical stack of input fields for various weather parameters: Location Code, Min Temp, Max Temp, Rainfall, Wind Gust Speed, Wind Speed 9am, Wind Speed 3pm, Humidity 9am, Humidity 3pm, Pressure 9am, Pressure 3pm, Temp 9am, Temp 3pm, a "Rain Today?" dropdown menu, Wind Gust Dir Code, Wind Dir 9am Code, Wind Dir 3pm Code, Year, Month, and Day. A "Predict" button is located at the bottom of the form.

Rainfall Prediction System

Location Code:

Min Temp:

Max Temp:

Rainfall:

Wind Gust Speed:

Wind Speed 9am:

Wind Speed 3pm:

Humidity 9am:

Humidity 3pm:

Pressure 9am:

Pressure 3pm:

Temp 9am:

Temp 3pm:

Rain Today?

Wind Gust Dir Code:

Wind Dir 9am Code:

Wind Dir 3pm Code:

Year:

Month:

Day:

2. chance.html

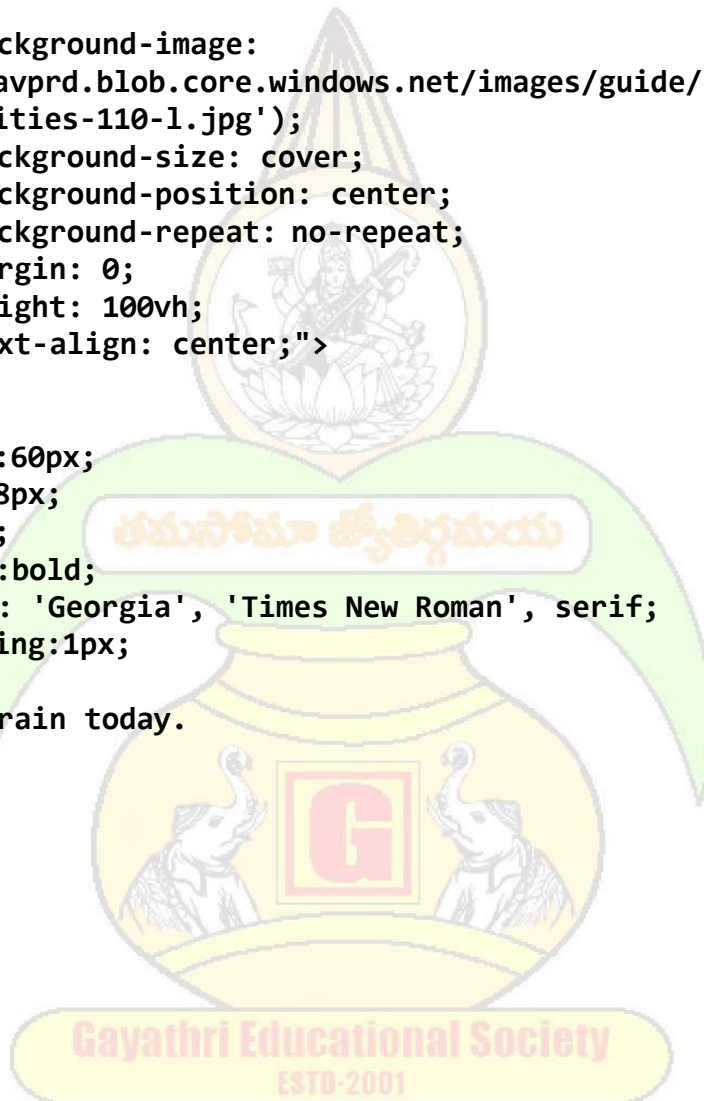
Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Rain Prediction</title>
</head>

<body style="background-image:
url('https://wnavprd.blob.core.windows.net/images/guide/cape-cod-
rainy-day-activities-110-1.jpg');
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      margin: 0;
      height: 100vh;
      text-align: center;">

  <h1 style="
padding-top:60px;
font-size:48px;
color:black;
font-weight:bold;
font-family: 'Georgia', 'Times New Roman', serif;
letter-spacing:1px;
">
    chances of rain today.
  </h1>

</body>
</html>
```



Output:



3. nochance.html

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Rain Prediction</title>
</head>

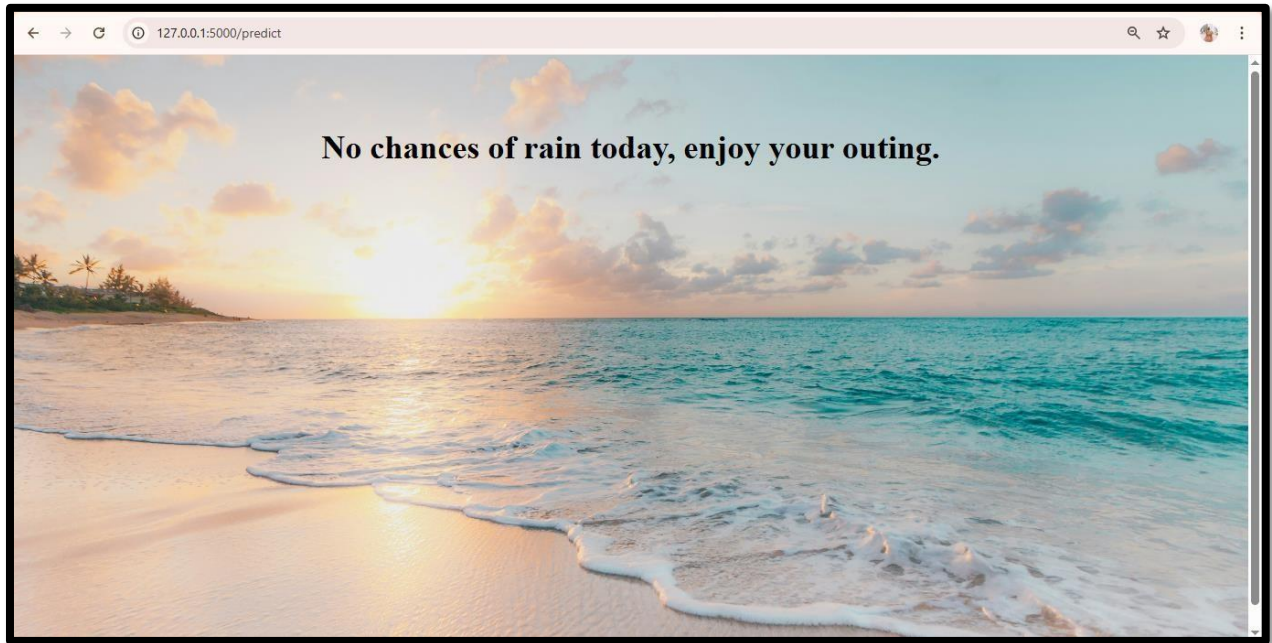
<body style="background-image:url('https://images.unsplash.com/photo-
1507525428034-b723cf961d3e');
  background-size:cover;
  background-position:center;
  background-repeat:no-repeat;
  margin:0;
  height:100vh;
  text-align:center;">

  <h1 style="padding-top:70px;
    font-size:45px;
    color:black;
    font-weight:bold;">
    No chances of rain today, enjoy your outing.
  </h1>
```

EXPLORATORY ANALYSIS OF RAINFALL DATA IN INDIA FOR AGRICULTURE

```
</body>  
</html>
```

Output:



12.2 - DATASET LINK:

Link:

https://docs.google.com/spreadsheets/d/1RA2000LZTeQyKl_mvnensAjp6LM4YzWI1Tz0SUG5-Ao/edit#gid=121883362

QR code:



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

12.3 - GitHub and Project Demo Link:

GitHub:

Link: <https://github.com/chitti4569/LTVIP2026TMIDS89121-Thota-Dhana-Sree>

QR Code:



Project Demo Link:

Link:

[https://drive.google.com/file/d/1OuIOD2VkHokn0Az-N6Ov0HiibGKjHVNh/view?usp=drive link](https://drive.google.com/file/d/1OuIOD2VkHokn0Az-N6Ov0HiibGKjHVNh/view?usp=drive_link)

QR Code:



Gayathri Educational Society
ESTD-2001